# Введение

Цель работы — ознакомление с методикой расчета оптимальных параметров регулятора при заданной структурной автоматической системы регулирования и заданных параметрах регулирования по интегральным критериям качества процесса регулирования.

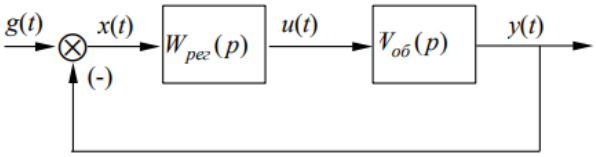


График 1: Структурная схема системы

Передаточная функция объекта:

$$W_{\text{об}(p)} = \frac{k_{\text{об}}}{T_{\text{об}}^2 p^2 + 2\zeta T_{\text{об}} p + 1} * \lambda^{-\tau_{\text{об}} p}$$

Передаточная функция регулятора:

$$W_{\text{per}(p)} = q_1 + \frac{q_2}{p} + q_3 p$$

Параметры объекта:

$$k_{\text{об}} = 1, \quad T_{\text{об}} = 1, \quad \zeta = 0,75, \quad \tau_{\text{об}} = 0,1$$

Критерий оптимизации:

$$I_3 = \int_0^\infty (x^2(t) + \mu^2 \hat{x}^2(t)) dt$$

# Ход работы

$W(p) = \frac{k}{T^2p^2+2\zeta Tp+1}$– колебательное звено

Распишем переход от передаточной функции к дифференциальному уравнению:

$$W(p) = \frac{k}{T^2p^2 + 2\zeta Tp + 1} = \frac{y}{g}$$

$$T^2\frac{d^2y}{dt^2} + 2T\zeta\frac{dy}{dt} + y = kg$$

Избавимся от коэффициента, тогда:

$$\frac{d^2y}{dt^2} + \frac{2\zeta}{T}\frac{dy}{dt} + \frac{y}{T^2} = \frac{k}{T^2}g$$

Коэффициенты для пространства состояний:

$$a_2 = 1, \quad a_1 = \frac{2\zeta}{T}, \quad a_0 = \frac{1}{T^2}, \quad b_0 = \frac{k}{T^2}, \quad b_1 = b_2 = 0, z_3 = 0$$

Подготовим звено к моделированию методом непосредственного интегрирования:

$$y = z_1 + b_2g = z_1$$

$$\frac{dz_1}{dt} = -(a_1y - b_1g) + z_2 = -\frac{2\zeta}{T}y + z_2$$

$$\frac{dz_2}{dt} = -(a_0y - b_0g) + z_3 = -\frac{y}{T^2} + \frac{k}{T^2}g$$

Путем преобразований исходное дифференциальное уравнение второго порядка сводится к эквивалентной системе из двух уравнений первого порядка:

$$\begin{cases} y = z_1 \\ \frac{dz_1}{dt} = -\frac{2\zeta}{T}y + z_2 \\ \frac{dz_2}{dt} = -\frac{y}{T^2} + \frac{k}{T^2}g \end{cases}$$

Применяя метод Рунге-Кутта получаем:

$$\begin{cases} x = g - y \quad y = z_1 \\ k_1 = \mathrm{dt}\ \left(z_2 - \frac{2\zeta}{T}y\right) \\ m_1 = \mathrm{dt}\ \left(\frac{k}{T^2}x - \frac{y}{T^2}\right) \\ k_2 = \mathrm{dt}\ \left(z_2 + \frac{m_1}{2} - \frac{2\zeta}{T}\left(y + \frac{k_1}{2}\right)\right) \\ m_2 = \mathrm{dt}\ \left(\frac{k}{T^2}x - \frac{1}{T^2}\left(y + \frac{k_1}{2}\right)\right) \\ k_3 = \mathrm{dt}\ \left(z_2 + \frac{m_2}{2} - \frac{2\zeta}{T}\left(y + \frac{k_2}{2}\right)\right) \\ m_3 = \mathrm{dt}\ \left(\frac{k}{T^2}x - \frac{1}{T^2}\left(y + \frac{k_2}{2}\right)\right) \\ k_4 = \mathrm{dt}\ \left(z_2 + m_3 - \frac{2\zeta}{T}(y + k_3)\right) \\ m_4 = \mathrm{dt}\ \left(\frac{k}{T^2}x - \frac{1}{T^2}(y + k_3)\right) \end{cases}$$

$$z_1 = z_1 + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

$$z_2 = z_2 + \frac{1}{6}(m_1 + 2m_2 + 2m_3 + m_4)$$

# Моделирование замкнутой системы с запаздыванием без ПИД-регулятора

График:

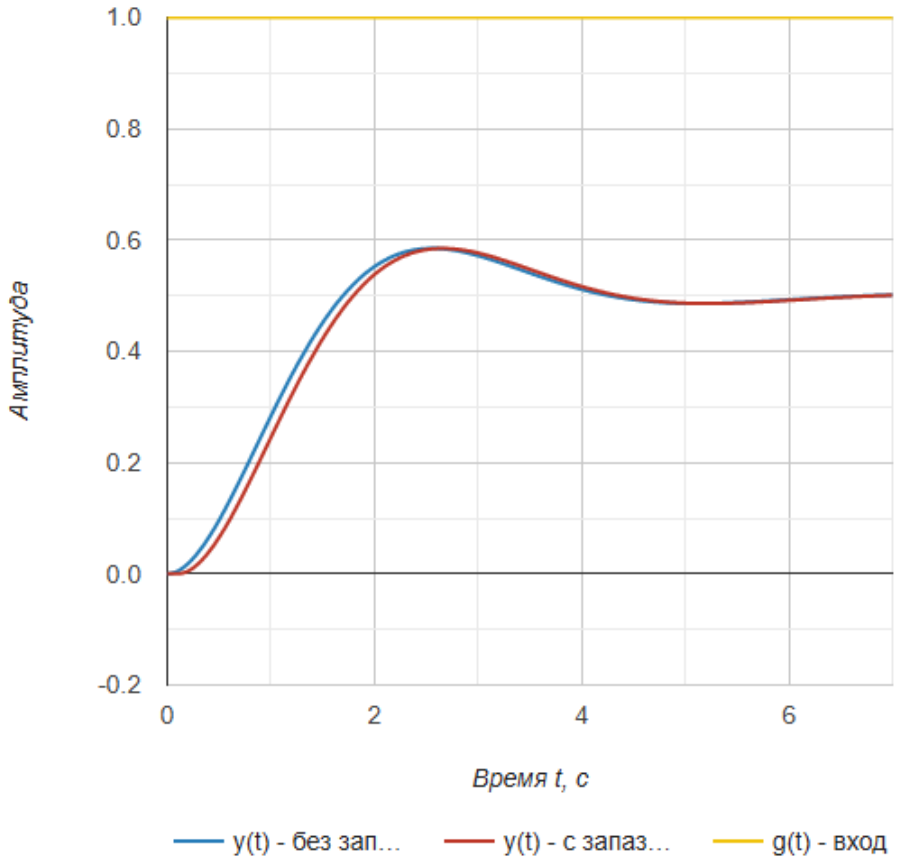


График 2: Переходный процесс без ПИД-регулятора

Смоделируем данную систему в SimInTech и сравним результаты графиков:

Структурная схема:

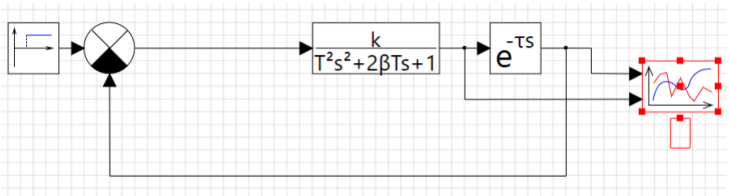


График 3: Структурная схема без ПИД-регулятора

График:

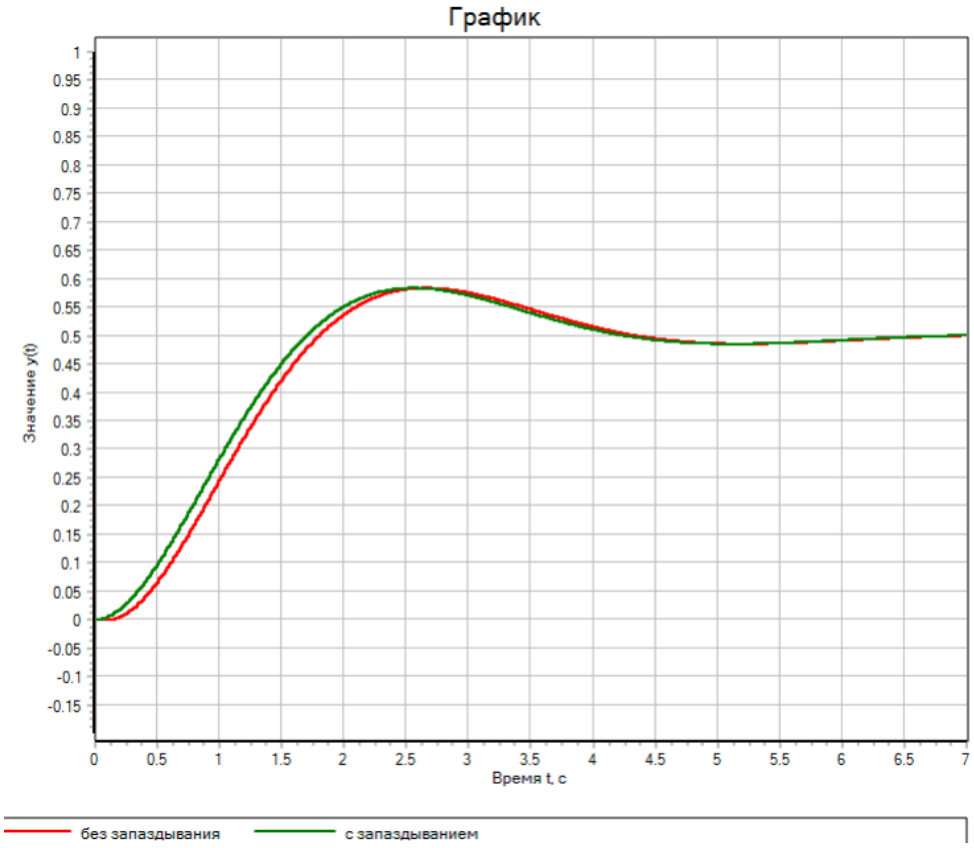


График 4: Переходный процесс в SimInTech

На графиках видно, что наша система имеет запаздывание: реакция на входной сигнал происходит не мгновенно, а с некоторой задержкой. Кроме того, если просто замкнуть систему обратной связью без регулятора, выходной сигнал не доходит до нужной нам отметки. Мы подаем на вход «ступеньку» (единицу), но система стабилизируется ниже, то есть остается ошибка.

Чтобы это исправить, в систему нужно добавить ПИД-регулятор. Его задача — сравнивать реальный выход с тем, который мы хотим получить, и автоматически подправлять сигнал, чтобы со временем устранить ошибку и компенсировать влияние запаздывания. При этом нам не нужно менять сам объект, мы просто добавляем управляющее звено.

# Моделирование замкнутой системы с ПИД-регулятором

Передаточная функция регулятора:

$$W_{\text{рег}(p)} = q_1 + \frac{q_2}{p} + q_3 p$$

Уравнения системы:

$$u(t,q) = W_{\text{рег}(p)} x(t,q)$$

$$y(t,q) = W_{\text{об}(p)} u(t,q)$$

ПИД-регулятор во временной области:

$$u(t,q) = q_1 x(t,q) + q_2 \int x(t,q)dt + q_3 \frac{dx(t,q)}{dt}$$

Для подбора оптимальных параметров $q_1, q_2, q_3$ необходим критерий оптимизации. В качестве такого критерия воспользуемся составным интегральным функционалом:

$$I_3 = \int_0^\infty (x^2(t) + \mu^2 \hat{x}^2(t))dt$$

График при параметрах $q_1 = q_2 = q_3 = 1$:

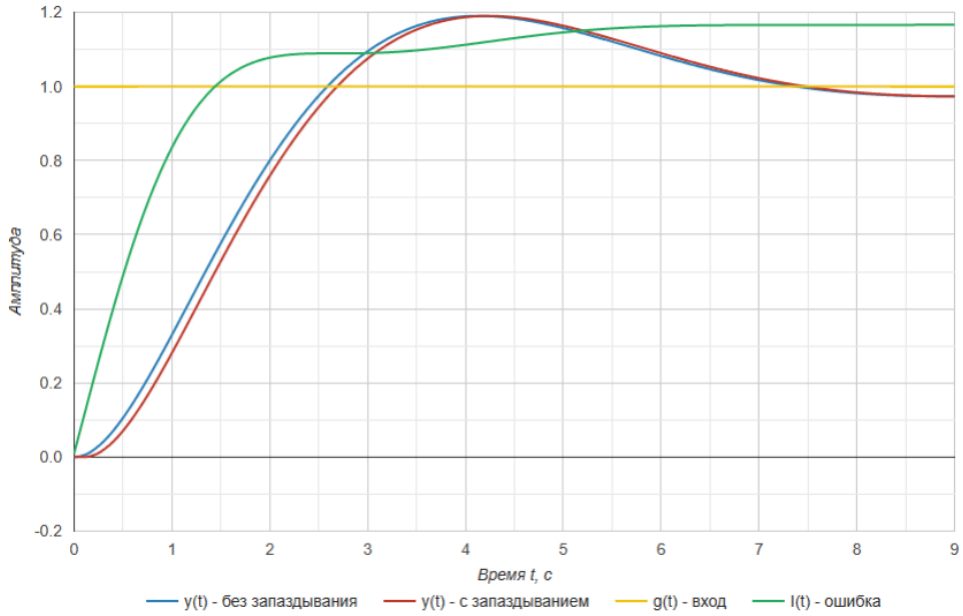


График 5: Переходный процесс с ПИД-регулятором (начальные параметры)

На графике видно, что при начальных параметрах регулятора ($q_1 = q_2 = q_3 = 1$) система выходит на заданное значение, однако качество переходного процесса остается низким. Значение интегральной ошибки составило $I = 1,1663$.

Это число будет использоваться нами как базовая точка для сравнения. Дальнейшая задача заключается в поиске таких коэффициентов ПИД-регулятора, при которых этот показатель ошибки станет минимальным, что будет означать оптимальную настройку системы.

# Подбор оптимальных параметров ПИД-регулятора методом покоординатного спуска

**Основная идея метода:** Покоординатный спуск — это итерационный метод оптимизации, при котором на каждом шаге оптимизируется только один параметр, а остальные фиксируются. Цикл повторяется до сходимости.

**Алгоритм:**

1. Инициализировать начальные значения параметров $q_1, q_2, q_3$ и шаг $h$.
2. Для каждого параметра $q_i$ ($i = 1, 2, 3$):
   1. Вычислить значение критерия $I$ при текущих значениях параметров.
   2. Увеличить $q_i$ на шаг $h$ и вычислить новый критерий $I_+$.
   3. Уменьшить $q_i$ на шаг $h$ (от начального значения) и вычислить критерий $I_-$.
   4. Если $I_+ < I$ и $I_+ < I_-$, принять $q_i = q_i + h$.
   5. Иначе, если $I_- < I$, принять $q_i = q_i - h$.
3. Если ни один параметр не изменился на шаге 2, уменьшить шаг: $h = \frac{h}{2}$.
4. Повторять шаги 2–3, пока шаг $h$ не станет меньше заданной точности $\varepsilon$.

Найдём оптимальные параметры:

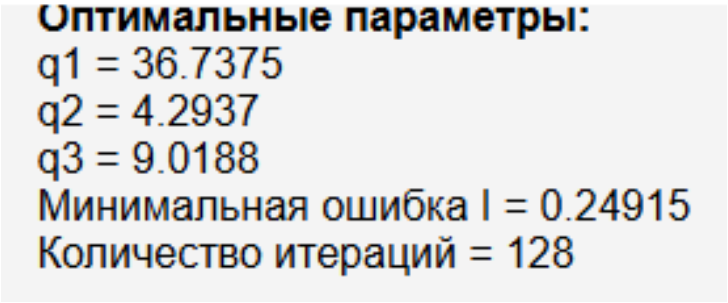


График 6: Оптимальные параметры ПИД-регулятора

График:

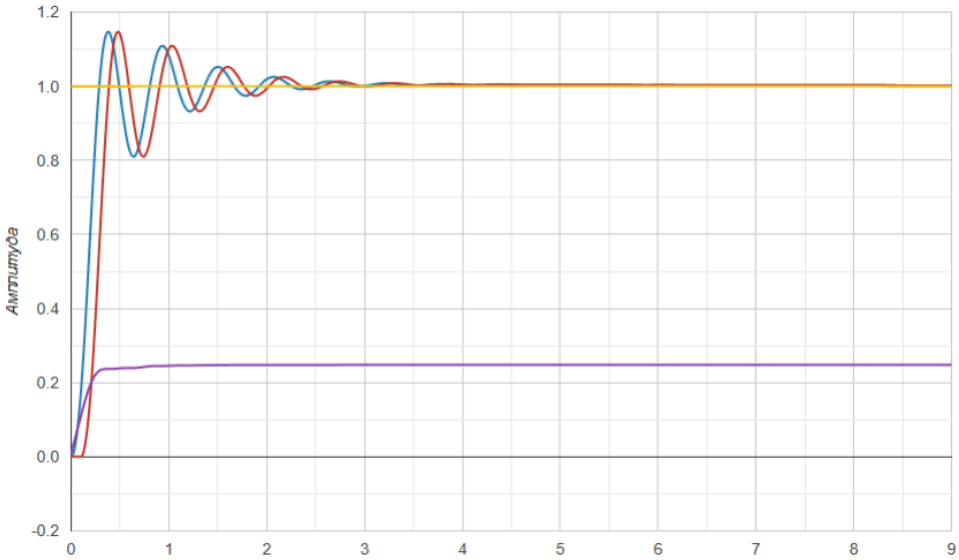


График 7: Переходный процесс с оптимальными параметрами

Сравним графики с реализацией в SimInTech:

Схема:

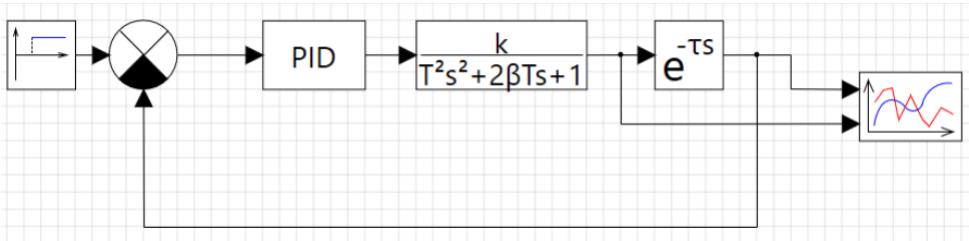


График 8: Структурная схема в SimInTech

График:

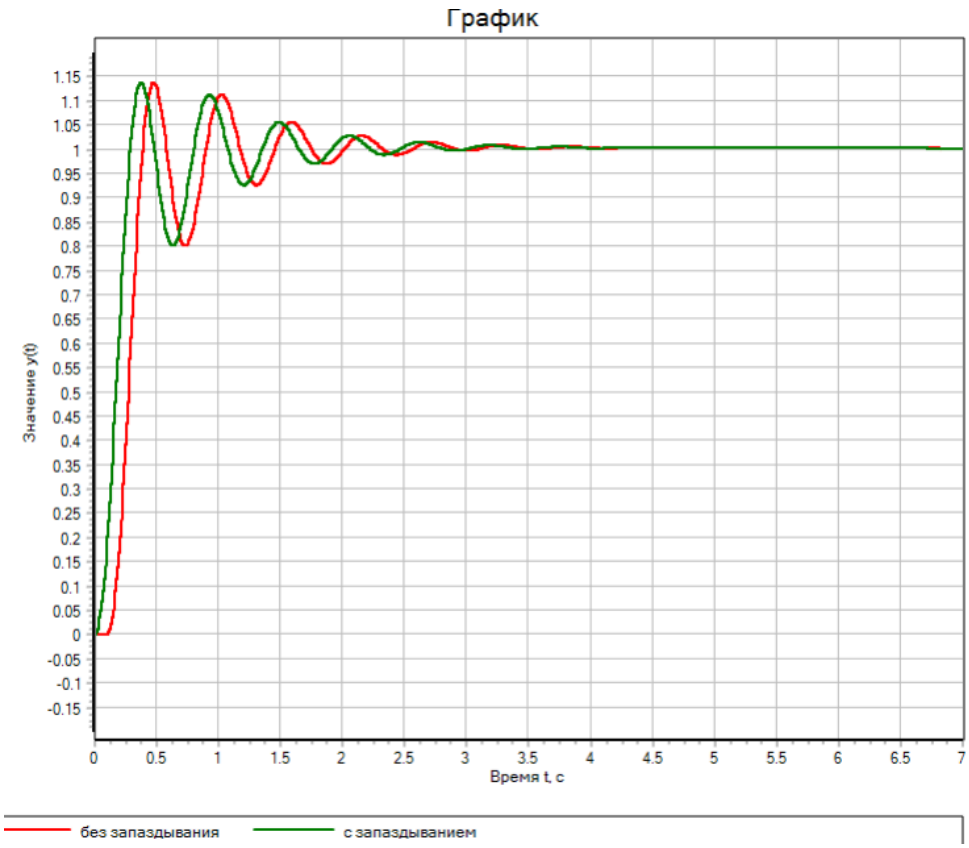


График 9: Переходный процесс в SimInTech

Применение метода покоординатного спуска дало заметный результат. На графиках видно, что переходный процесс стал гораздо качественнее по сравнению с начальной настройкой: система быстрее выходит на заданный уровень, а разрыв между желаемым и реальным значением сокращается значительно эффективнее.

Также уменьшилась интегральная ошибка $I$ почти в 12 раз (с $2,85$ до $0,24915$). Визуально это подтверждается тем, что кривая на графике теперь плотнее прилегает к линии уставки $y = 1$. Полное совпадение графиков, полученных при расчете и в среде SimInTech, доказывает, что алгоритм оптимизации отработал корректно.

# Расчёт функции чувствительности

При реализации алгоритма параметрической оптимизации градиентным методом необходимы значения каждой составляющей вектор-градиента целевой функции

$$\frac{\partial I(x(t,q))}{\partial q_j}$$

В общем виде составляющие градиента имеют вид:

$$\frac{\partial I}{\partial q_j} = \int_0^\infty \frac{\partial F(x(t,q),t)}{\partial x} \cdot \frac{\partial x(t,q)}{\partial q_j} dt$$

Учитывая, что для исследуемой системы ошибка определяется как $x(t,q) = g(t) - y(t,q)$, а входной сигнал не зависит от параметров регулятора, получим:

$$\frac{\partial x(t,q)}{\partial q_j} = \frac{\partial (g(t) - y(t,q))}{\partial q_j} = -\frac{\partial y(t,q)}{\partial q_j} = -\xi_{j(t)}$$

Здесь $\xi_j = \frac{\partial y(t,q)}{\partial q_j}$ — называется функцией чувствительности, которая характеризует влияние $j$-го настраиваемого параметра $q_j$ на выходную координату объекта регулирования.

Таким образом, для вычисления градиента необходимо предварительно вычислить три функции чувствительности $\xi_j$, так как ПИД-регулятор имеет три настраиваемых параметра ($j = 1, 2, 3$).

В соответствии со структурной схемой исследуемой системы:

$$y(t,q) = W_{\text{об}(p)} u(t,q)$$

$$u(t,q) = W_{\text{рег}(p)} x(t) = q_1 x(t) + \frac{q_2}{p} x(t) + q_3 p x(t)$$

Дифференцируя частным образом правую и левую части и учитывая независимость $W_{\text{об}(p)}$ от настраиваемых параметров $q_j$, получим:

$$\xi_j = \frac{\partial y(t,q)}{\partial q_j} = W_{\text{об}(p)} \frac{\partial u(t,q)}{\partial q_j}, \quad j = 1, 2, 3$$

Т.е. функция чувствительности представляет собой выходную координату модели объекта регулирования с входной координатой $\frac{\partial u(t,q)}{\partial q_j}$.

Таким образом, для вычисления значений функций чувствительности $\xi_j$ необходимо подать на вход объекта регулирования сигнал, соответствующий $j$-ой частной производной управляющего сигнала $\frac{\partial u(t,q)}{\partial q_j}$.

Для ПИД-регулятора $u(t,q) = \left(q_1 + \frac{q_2}{p} + q_3 p\right) x(t)$ частные производные управляющего сигнала имеют вид:

$$\begin{cases} \frac{\partial u}{\partial q_1} = x(t) - q_1 \xi_1(t) - \frac{q_2}{p} \xi_1(t) - q_3 p \xi_1(t) \\ \frac{\partial u}{\partial q_2} = \frac{1}{p} x(t) - q_1 \xi_2(t) - \frac{q_2}{p} \xi_2(t) - q_3 p \xi_2(t) \\ \frac{\partial u}{\partial q_3} = p x(t) - q_1 \xi_3(t) - \frac{q_2}{p} \xi_3(t) - q_3 p \xi_3(t) \end{cases}$$

Полученные сигналы $\frac{\partial u}{\partial q_j}$ подаются на вход модели объекта регулирования $W_{\text{об}(p)}$, с выхода которой снимаются значения функций чувствительности $\xi_1(t), \xi_2(t), \xi_3(t)$.

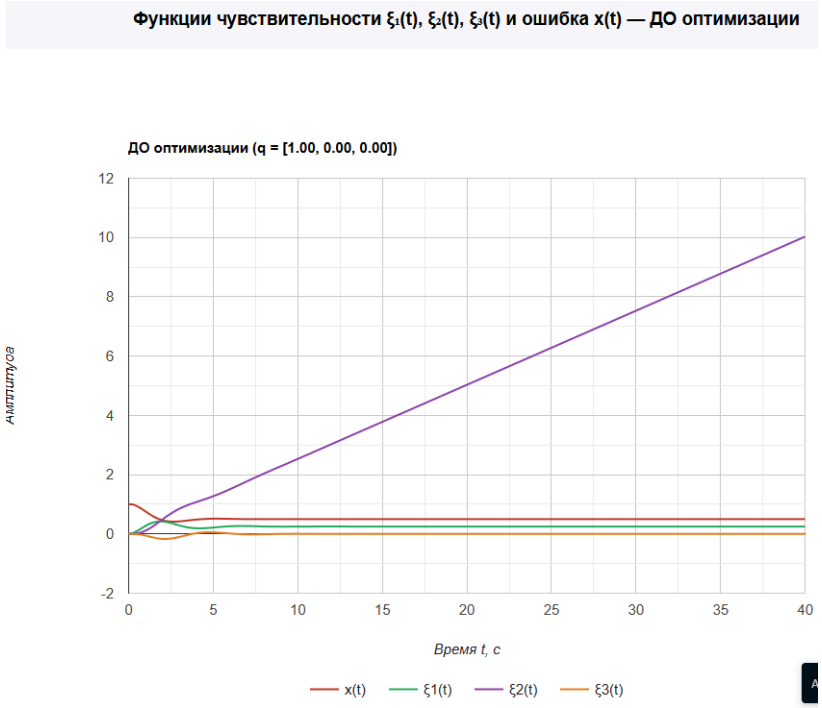


График 10: Функции чувствительности $\xi_1(t)$, $\xi_2(t)$, $\xi_3(t)$ и ошибка x(t) ДО оптимизации

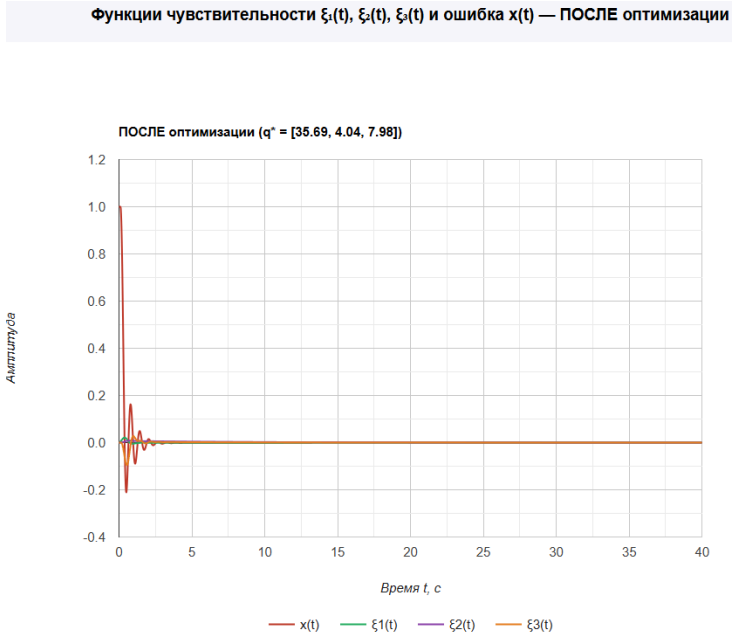


График 11: Функции чувствительности $\xi_1(t)$, $\xi_2(t)$, $\xi_3(t)$ и ошибка x(t) ПОСЛЕ оптимизации

На первом графике (до оптимизации) видно, что функции чувствительности имеют большой размах и долго не затухают, так как система работает нестабильно и ошибка $x(t)$ велика. На втором графике (после оптимизации) картина меняется: ошибка $x(t)$ прижимается к нулю гораздо быстрее, и функции чувствительности вслед за ней тоже быстро затухают. Это говорит о том, что система стала менее чувствительна к отклонениям параметров, так как мы уже нашли их оптимальные значения, при которых процесс завершается максимально быстро и точно.

# Алгоритм градиентного спуска с нормализацией и адаптивным шагом

**Основная идея метода:** Алгоритм ищет минимум, двигаясь строго против градиента. Чтобы избежать резких скачков параметров на крутых участках, вектор градиента нормализуется — так мы получаем только направление, а длину шага $h$ контролируем сами.

**Алгоритм:**

1. Задаются начальные значения параметров регулятора $q^0 = [q_1^0, q_2^0, q_3^0]$, начальный шаг $h^0 > 0$, точность по градиенту $\varepsilon > 0$ и минимально допустимый шаг $h_{\min}$. Полагаем номер итерации $l = 0$.

2. Для текущих параметров $q^l$ численно решается система дифференциальных уравнений. Определяются:
   - ошибка регулирования $x(t, q^l)$;
   - функции чувствительности $\xi_j(t, q^l)$ (с учетом запаздывания $\tau$);
   - текущее значение критерия качества $I(q^l) = \int_0^{T_{\text{мод}}} x^2(t, q^l)dt$.

3. Вычисляются компоненты градиента:

$$\frac{\partial I}{\partial q_j} = -2\int_0^{T_{\text{мод}}} x(t, q^l) \cdot \xi_j(t, q^l)dt, \quad j = 1, 2, 3$$

   Вычисляется евклидова норма градиента:

$$\|\nabla I\| = \sqrt{\sum_{j=1}^{3}\left(\frac{\partial I}{\partial q_j}\right)^2}$$

   Если $\|\nabla I\| < \varepsilon$, алгоритм завершается.

4. Вычисляются новые значения параметров (“пробная точка”):

$$q_j^{try} = q_j^l - h^l \cdot \frac{\frac{\partial I}{\partial q_j}}{\|\nabla I\|}, \quad j = 1, 2, 3$$

5. Проводится моделирование системы с параметрами $q^{try}$ и вычисляется новый критерий $I(q^{try})$.
   - Если $I(q^{try}) < I(q^l)$ (Успех): Принимаем новые параметры: $q^{l+1} = q^{try}$. Увеличиваем шаг для следующей итерации: $h^{l+1} = h^l \cdot 1.05$. Переходим к Шагу 2 с $l = l + 1$.
   - Если $I(q^{try}) \geq I(q^l)$ (Неудача): Параметры не обновляются: $q^{l+1} = q^l$. Уменьшаем шаг: $h^l = h^l \cdot 0.5$. Если $h^l < h_{\min}$, алгоритм завершается. В противном случае возвращаемся к Шагу 4 для повторного совершения пробного шага из той же точки.

**Результаты моделирования:**

**Оптимальные параметры:** q* = [35.6860, 4.0382, 7.9786]
**Критерий:** I* = 0.2503
**Градиент:** [-0.000330, -0.000029, -0.000031]
**Норма градиента:** ||∇I|| = 0.000332

График 12: Оптимальные параметры ПИД-регулятора, полученные методом градиентного спуска с нормализацией

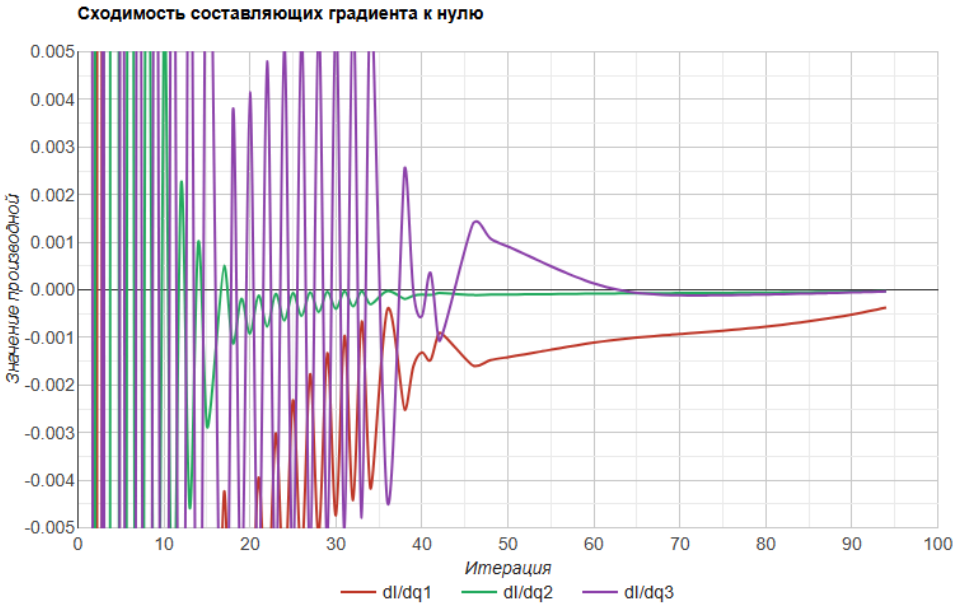


График 13: Сходимость составляющих градиента к нулю

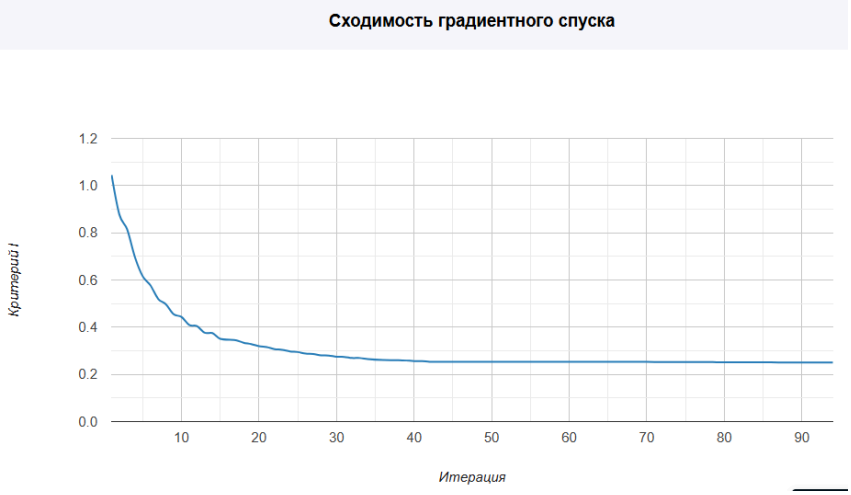


График 14: Сходимость градиентного спуска

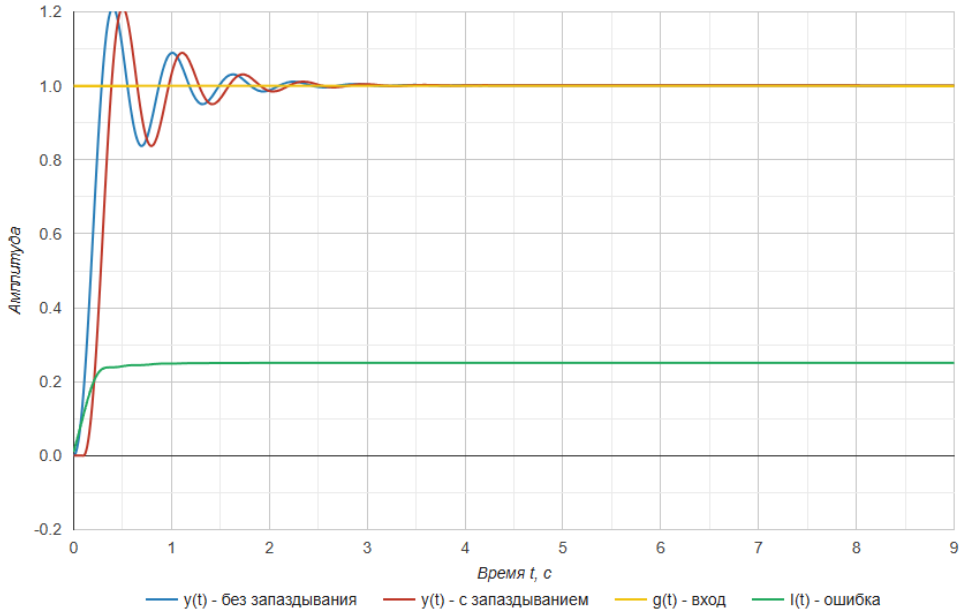


График 15: Переходный процесс с оптимальными параметрами ($q_1$ = 35.686, $q_2$ = 4.0382, $q_3$ = 7.9786)

По графикам видно, что алгоритм нашёл минимум: градиент быстро вышел к нулю, а критерий снизился с 2,85 до 0,2503. С полученными параметрами система выходит на уставку за  3 с небольшим перерегулированием. Этот результат практически совпадает с покоординатным спуском (I = 0,24915), что говорит о правильности моделирования.

# Алгоритм наискорейшего спуска с линейным поиском ($I_3(2)$)

**Основная идея метода:** Метод наискорейшего спуска — это градиентный метод оптимизации, в котором на каждой итерации движение осуществляется в направлении антиградиента (направлении наибыстрейшего убывания функционала). Ключевое отличие от простого градиентного спуска — наличие внутреннего цикла линейного поиска: шаг в выбранном направлении выполняется до тех пор, пока функционал уменьшается.

**Критерий оптимизации ($I_3(2)$):** В отличие от стандартного критерия $I_3 = \int x^2(t)dt$, используется модифицированный функционал, минимизирующий отклонение от эталонной модели:

$$I_3^{\{(2)\}} = \int_0^L \left(y(t) - y_{\text{эт}(t)}\right)^2 dt$$

где $y(t)$ — выходной сигнал оптимизируемой системы, $y_{\text{эт}(t)}$ — выход эталонной модели.

**Эталонная модель:** Инерционное звено первого порядка:

$$W_{\text{эт}(p)} = \frac{k_{\text{эт}}}{T_{\text{эт}}p + 1}$$

с параметрами $k_{\text{эт}} = 1$, $T_{\text{эт}} = 0.6$.

Расчёт выхода эталонной модели производится численно методом Рунге-Кутты 4-го порядка:

$$\begin{cases} k_1 = \text{dt}\left(-\frac{y_{\text{эт}}}{T_{\text{эт}}} + \frac{k_{\text{эт}}}{T_{\text{эт}}} * g\right) \\ k_2 = \text{dt}\left(-\frac{y_{\text{эт}} + \frac{k_1}{2}}{T_{\text{эт}}} + \frac{k_{\text{эт}}}{T_{\text{эт}}} * g\right) \\ k_3 = \text{dt}\left(-\frac{y_{\text{эт}} + \frac{k_2}{2}}{T_{\text{эт}}} + \frac{k_{\text{эт}}}{T_{\text{эт}}} * g\right) \\ k_4 = \text{dt}\left(-\frac{y_{\text{эт}} + k_3}{T_{\text{эт}}} + \frac{k_{\text{эт}}}{T_{\text{эт}}} * g\right) \end{cases}$$

$$y_{\text{эт}} = y_{\text{эт}} + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

**Градиент функционала:** Для критерия $I_3(2)$ градиент вычисляется по формуле:

$$\frac{\partial I}{\partial q_j} = 2\int_0^L \left(y(t) - y_{\text{эт}(t)}\right) \cdot \xi_{j(t)} dt, \quad j = 1, 2, 3$$

где $\xi_{j(t)} = \frac{\partial y(t)}{\partial q_j}$ — функции чувствительности.

**Алгоритм:**

1. Задаются начальные значения параметров регулятора $q^0 = [q_1^0, q_2^0, q_3^0]$, начальный шаг $h^0 > 0$, точность по градиенту $\varepsilon = 10^{-4}$. Полагаем общий счётчик итераций $m = 0$, максимальное число итераций $m_{\max} = 200$.

2. Для текущих параметров $q$ численно решается система дифференциальных уравнений. Определяются:
   - ошибка регулирования $x(t, q) = g(t) - y(t, q)$;
   - выход эталонной модели $y_{\text{эт}(t)}$;
   - разность $\Delta(t) = y(t, q) - y_{\text{эт}(t)}$;

- функции чувствительности $\xi_j(t,q)$ (с учетом запаздывания $\tau$);
- текущее значение критерия качества $I(q) = \int_0^L \Delta^2(t)dt$;
- компоненты градиента

$$\frac{\partial I}{\partial q_j} = 2\int_0^L \Delta(t)\cdot\xi_j(t,q)dt, \quad j = 1,2,3$$

.

3. Вычисляется евклидова норма градиента:

$$\|\nabla I\| = \sqrt{\sum_{j=1}^{3}\left(\frac{\partial I}{\partial q_j}\right)^2}$$

Если $\|\nabla I\| < \varepsilon$, алгоритм завершается.

4. Определяется направление наискорейшего спуска (единичный вектор антиградиента):

$$S = -\frac{\nabla I}{\|\nabla I\|}, \quad \text{т.е.} \quad S_j = -\frac{\left(\frac{\partial I}{\partial q_j}\right)}{\|\nabla I\|}, \quad j = 1,2,3$$

5. Линейный поиск вдоль направления $S$:
   1. Полагаем $q_{\text{лучш}} = q$, $I_{\text{лучш}} = I(q)$, $h_{\text{тек}} = h$, счётчик шагов $n = 0$.
   2. В цикле (максимум 50 шагов):
      1. Вычислить пробные параметры: $q_{\text{проб}} = q + h_{\text{тек}}\cdot S$, с ограниждением $q_{\text{проб}} = \max\left(0, q_{\text{проб}}\right)$.
      2. Вычислить $I\left(q_{\text{проб}}\right)$.
      3. Если $I\left(q_{\text{проб}}\right) < I_{\text{лучш}}$ (успешный шаг):
         1. Принять $q_{\text{лучш}} = q_{\text{проб}}$, $I_{\text{лучш}} = I\left(q_{\text{проб}}\right)$, $q = q_{\text{проб}}$.
         2. Увеличить шаг: $h_{\text{тек}} = h_{\text{тек}}\cdot 1.4$.
         3. Увеличить счётчики: $n = n+1$, $m = m+1$.
         4. Записать точку в историю: параметры $q$, критерий $I$, градиент не фиксируется (внутринаправленный шаг).
         5. Продолжить цикл.
      4. Иначе (неудача, перелёт через минимум):
         1. Вернуть $q = q_{\text{лучш}}$, $I = I_{\text{лучш}}$.
         2. Пересчитать градиент $\nabla I(q)$ для выбора нового направления.
         3. Записать точку в историю с новым градиентом.
         4. Увеличить $m = m+1$.
         5. Выйти из цикла линейного поиска.

6. Если на шаге 5 не было сделано ни одного успешного шага ($n = 0$):
   1. Уменьшить начальный шаг: $h = \frac{h}{2}$.
   2. Если $h < 10^{-3}$, завершить алгоритм.

   Иначе сбросить шаг к начальному: $h = h^0$.

7. Если $m \geq m_{\max}$, завершить алгоритм. Вернуться к шагу 2.

**Код реализации:**

```
<!doctype html>
<html>
```

```html
<head>
  <meta charset="utf-8" />
  <script
    type="text/javascript"
    src="https://www.gstatic.com/charts/loader.js"
  ></script>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 0;
      padding: 20px;
      background: #f5f6fa;
    }
    .container {
      display: flex;
      justify-content: center;
      margin: 20px 0;
    }
    #result_info {
      background: #fff;
      padding: 15px;
      border-radius: 8px;
      box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
      max-width: 800px;
      margin: 20px auto;
      text-align: center;
      font-size: 16px;
    }
  </style>
  <script type="text/javascript">
    google.charts.load("current", { packages: ["corechart"] });
    google.charts.setOnLoadCallback(drawChart);

    // Симуляция: считает I и градиент по методу чувствительности
    function simulate(q1, q2, q3) {
      // Параметры объекта: T, k, zeta, tau — запаздывание
      var T = 1,
        k = 1,
        zeta = 0.75,
        tau = 0.1,
        g = 1,
        dt = 0.01,
        L = 50;
      // Переменные состояния объекта
      var z1 = 0,
        z2 = 0;
      // Ошибка, интеграл, производная, управление
      var x,
        x_prev = g,
        intx = 0,
        dx,
        u;
      var I = 0;
      // Функции чувствительности: eta = ξ, nu = dξ/dt
      var eta1 = 0,
        eta2 = 0,
```

```
  eta3 = 0;
var nu1 = 0,
  nu2 = 0,
  nu3 = 0;
// Интегралы от функций чувствительности
var int_xi1 = 0,
  int_xi2 = 0,
  int_xi3 = 0;
// Компоненты градиента
var gradI1 = 0,
  gradI2 = 0,
  gradI3 = 0;
var data = [],
  gradData = [],
  gradIData = [];
var t_ = 0;

// Эталонная модель: инерционное звено 1-го порядка
var T_ref = 0.6,
  k_ref = 1;
var y_ref_prev = 0;

// Буфер запаздывания
var delaySteps = Math.max(1, Math.round(tau / dt));
var historyY = new Array(delaySteps).fill(0);
var historyXi1 = new Array(delaySteps).fill(0);
var historyXi2 = new Array(delaySteps).fill(0);
var historyXi3 = new Array(delaySteps).fill(0);

while (t_ <= L) {
  var y_now = z1;

  // Сдвигаем буфер — достаём задержанные значения
  var y_delayed = historyY.shift();
  var xi1_delayed = historyXi1.shift();
  var xi2_delayed = historyXi2.shift();
  var xi3_delayed = historyXi3.shift();

  // ПИД-регулятор по задержанному сигналу
  x = g - y_delayed;
  intx += x * dt;
  dx = (x - x_prev) / dt;
  x_prev = x;
  u = q1 * x + q2 * intx + q3 * dx;

  // Эталонная модель — РК4
  var obrat_g = g;
  var k1_ref = dt * (-(y_ref_prev / T_ref) + (k_ref / T_ref) * obrat_g);
  var k2_ref =
    dt *
    (-(y_ref_prev + k1_ref / 2) / T_ref + (k_ref / T_ref) * obrat_g);
  var k3_ref =
    dt *
    (-(y_ref_prev + k2_ref / 2) / T_ref + (k_ref / T_ref) * obrat_g);
  var k4_ref =
    dt * (-(y_ref_prev + k3_ref) / T_ref + (k_ref / T_ref) * obrat_g);
```

```
var y_ref =
  y_ref_prev + (1 / 6) * (k1_ref + 2 * k2_ref + 2 * k3_ref + k4_ref);
y_ref_prev = y_ref;

// Ошибка относительно эталона
var diff = y_delayed - y_ref;

// Интеграл от квадрата ошибки
I += diff * diff * dt;

// Сдвигаем буфер — добавляем текущие значения
historyY.push(y_now);
historyXi1.push(eta1);
historyXi2.push(eta2);
historyXi3.push(eta3);

// Производные управления по параметрам q1, q2, q3
var dxi1_dt = nu1,
  dxi2_dt = nu2,
  dxi3_dt = nu3;

var du_dq1 = x - q1 * xi1_delayed - q2 * int_xi1 - q3 * dxi1_dt;
var du_dq2 = intx - q1 * xi2_delayed - q2 * int_xi2 - q3 * dxi2_dt;
var du_dq3 = dx - q1 * xi3_delayed - q2 * int_xi3 - q3 * dxi3_dt;

// Уравнения чувствительности: deta/dt = nu, dnu/dt = (k/T²)·du/dq - (2ζ/
T)·nu - (1/T²)·eta

// РК4 для чувствительности ξ1 (eta1, nu1)
var eta1_k1 = nu1 * dt;
var nu1_k1 =
  ((k / (T * T)) * du_dq1 -
    ((2 * zeta) / T) * nu1 -
    (1 / (T * T)) * eta1) *
  dt;

var eta1_k2 = (nu1 + nu1_k1 / 2) * dt;
var nu1_k2 =
  ((k / (T * T)) * du_dq1 -
    ((2 * zeta) / T) * (nu1 + nu1_k1 / 2) -
    (1 / (T * T)) * (eta1 + eta1_k1 / 2)) *
  dt;

var eta1_k3 = (nu1 + nu1_k2 / 2) * dt;
var nu1_k3 =
  ((k / (T * T)) * du_dq1 -
    ((2 * zeta) / T) * (nu1 + nu1_k2 / 2) -
    (1 / (T * T)) * (eta1 + eta1_k2 / 2)) *
  dt;

var eta1_k4 = (nu1 + nu1_k3) * dt;
var nu1_k4 =
  ((k / (T * T)) * du_dq1 -
    ((2 * zeta) / T) * (nu1 + nu1_k3) -
    (1 / (T * T)) * (eta1 + eta1_k3)) *
  dt;
```

```
eta1 = eta1 + (eta1_k1 + 2 * eta1_k2 + 2 * eta1_k3 + eta1_k4) / 6;
nu1 = nu1 + (nu1_k1 + 2 * nu1_k2 + 2 * nu1_k3 + nu1_k4) / 6;

// РК4 для ξ2 (eta2, nu2)
var eta2_k1 = nu2 * dt;
var nu2_k1 =
  ((k / (T * T)) * du_dq2 -
    ((2 * zeta) / T) * nu2 -
    (1 / (T * T)) * eta2) *
  dt;

var eta2_k2 = (nu2 + nu2_k1 / 2) * dt;
var nu2_k2 =
  ((k / (T * T)) * du_dq2 -
    ((2 * zeta) / T) * (nu2 + nu2_k1 / 2) -
    (1 / (T * T)) * (eta2 + eta2_k1 / 2)) *
  dt;

var eta2_k3 = (nu2 + nu2_k2 / 2) * dt;
var nu2_k3 =
  ((k / (T * T)) * du_dq2 -
    ((2 * zeta) / T) * (nu2 + nu2_k2 / 2) -
    (1 / (T * T)) * (eta2 + eta2_k2 / 2)) *
  dt;

var eta2_k4 = (nu2 + nu2_k3) * dt;
var nu2_k4 =
  ((k / (T * T)) * du_dq2 -
    ((2 * zeta) / T) * (nu2 + nu2_k3) -
    (1 / (T * T)) * (eta2 + eta2_k3)) *
  dt;

eta2 = eta2 + (eta2_k1 + 2 * eta2_k2 + 2 * eta2_k3 + eta2_k4) / 6;
nu2 = nu2 + (nu2_k1 + 2 * nu2_k2 + 2 * nu2_k3 + nu2_k4) / 6;

// РК4 для ξ3 (eta3, nu3)
var eta3_k1 = nu3 * dt;
var nu3_k1 =
  ((k / (T * T)) * du_dq3 -
    ((2 * zeta) / T) * nu3 -
    (1 / (T * T)) * eta3) *
  dt;

var eta3_k2 = (nu3 + nu3_k1 / 2) * dt;
var nu3_k2 =
  ((k / (T * T)) * du_dq3 -
    ((2 * zeta) / T) * (nu3 + nu3_k1 / 2) -
    (1 / (T * T)) * (eta3 + eta3_k1 / 2)) *
  dt;

var eta3_k3 = (nu3 + nu3_k2 / 2) * dt;
var nu3_k3 =
  ((k / (T * T)) * du_dq3 -
    ((2 * zeta) / T) * (nu3 + nu3_k2 / 2) -
    (1 / (T * T)) * (eta3 + eta3_k2 / 2)) *
```

```
      dt;

    var eta3_k4 = (nu3 + nu3_k3) * dt;
    var nu3_k4 =
      ((k / (T * T)) * du_dq3 -
        ((2 * zeta) / T) * (nu3 + nu3_k3) -
        (1 / (T * T)) * (eta3 + eta3_k3)) *
      dt;

    eta3 = eta3 + (eta3_k1 + 2 * eta3_k2 + 2 * eta3_k3 + eta3_k4) / 6;
    nu3 = nu3 + (nu3_k1 + 2 * nu3_k2 + 2 * nu3_k3 + nu3_k4) / 6;

    int_xi1 += xi1_delayed * dt;
    int_xi2 += xi2_delayed * dt;
    int_xi3 += xi3_delayed * dt;

    // Градиент: ∂I/∂qj = 2 ∫ (y - y_эт) · ξj dt
    gradI1 += 2 * diff * xi1_delayed * dt;
    gradI2 += 2 * diff * xi2_delayed * dt;
    gradI3 += 2 * diff * xi3_delayed * dt;

    data.push([t_, x, eta1, eta2, eta3, y_ref, y_delayed]);
    gradData.push([
      t_,
      2 * diff * xi1_delayed,
      2 * diff * xi2_delayed,
      2 * diff * xi3_delayed,
    ]);
    gradIData.push([t_, gradI1, gradI2, gradI3]);

    // РК4 для объекта (2-й порядок)
    var k1 = dt * (z2 - ((2 * zeta) / T) * y_now);
    var m1 = dt * ((k / (T * T)) * u - y_now / (T * T));
    var k2 = dt * (z2 + m1 / 2 - ((2 * zeta) / T) * (y_now + k1 / 2));
    var m2 = dt * ((k / (T * T)) * u - (1 / (T * T)) * (y_now + k1 / 2));
    var k3 = dt * (z2 + m2 / 2 - ((2 * zeta) / T) * (y_now + k2 / 2));
    var m3 = dt * ((k / (T * T)) * u - (1 / (T * T)) * (y_now + k2 / 2));
    var k4 = dt * (z2 + m3 - ((2 * zeta) / T) * (y_now + k3));
    var m4 = dt * ((k / (T * T)) * u - (1 / (T * T)) * (y_now + k3));

    z1 += (1 / 6) * (k1 + 2 * k2 + 2 * k3 + k4);
    z2 += (1 / 6) * (m1 + 2 * m2 + 2 * m3 + m4);
    t_ += dt;
  }
  return {
    I: I,
    grad: [gradI1, gradI2, gradI3],
    data: data,
    gradData: gradData,
    gradIData: gradIData,
  };
}

// Наискорейший спуск: антиградиент + линейный поиск с адаптивным шагом
function gradientNorm(grad) {
  return Math.sqrt(
```

```
    grad[0] * grad[0] + grad[1] * grad[1] + grad[2] * grad[2],
  );
}

function runSteepestDescent(qStart, hInit, maxIter) {
  var q = qStart.slice();
  var h = hInit;

  // История для графиков
  var history_q = [];
  var history_I = [];
  var history_grad = [];

  var totalIter = 0;

  // Начальная точка — считаем градиент
  var currentResult = simulate(q[0], q[1], q[2]);
  var currentI = currentResult.I;
  var currentGrad = currentResult.grad.slice();

  // Сохраняем начальную точку
  history_q.push({ q1: q[0], q2: q[1], q3: q[2] });
  history_I.push(currentI);
  history_grad.push({
    g1: currentGrad[0],
    g2: currentGrad[1],
    g3: currentGrad[2],
  });

  while (totalIter < maxIter) {
    var grad = currentResult.grad;
    var norm = gradientNorm(grad);

    // Проверка сходимости: норма градиента < 1e-4
    if (norm < 1e-4) break;

    // Направление — антиградиент, нормированный
    var S = [-grad[0] / norm, -grad[1] / norm, -grad[2] / norm];

    // Линейный поиск вдоль S
    var qBest = q.slice();
    var IBest = currentI;
    var qStartBest = q.slice();

    var stepCount = 0;
    var hCurrent = h;

    while (stepCount < 50) {
      var qNext = [];
      for (var i = 0; i < 3; i++) {
        qNext.push(Math.max(0.0, q[i] + hCurrent * S[i]));
      }

      var nextResult = simulate(qNext[0], qNext[1], qNext[2]);
      var nextI = nextResult.I;
```

```
    if (nextI < IBest) {
      // Успех — двигаемся, увеличиваем шаг
      qBest = qNext.slice();
      IBest = nextI;
      q = qNext.slice();
      hCurrent *= 1.4;
      stepCount++;
      totalIter++;

      // Градиент = null — это внутренаправленный шаг
      history_q.push({ q1: q[0], q2: q[1], q3: q[2] });
      history_I.push(IBest);
      history_grad.push(null);
    } else {
      // Неудача — откат, пересчёт градиента для нового направления
      q = qBest.slice();
      currentI = IBest;

      history_q[history_q.length - 1] = {
        q1: q[0],
        q2: q[1],
        q3: q[2],
      };
      history_I[history_I.length - 1] = currentI;

      currentResult = simulate(q[0], q[1], q[2]);
      currentGrad = currentResult.grad.slice();
      history_grad[history_grad.length - 1] = {
        g1: currentGrad[0],
        g2: currentGrad[1],
        g3: currentGrad[2],
      };

      totalIter++;
      break;
    }
  }

  // Ни одного успеха — уменьшаем шаг
  if (stepCount === 0) {
    h *= 0.5;
    if (h < 1e-3) break;
  } else {
    h = hInit;
  }
}

return {
  q: q,
  I: currentI,
  grad: currentGrad,
  history_q: history_q,
  history_grad: history_grad,
  history_I: history_I,
  totalIter: totalIter,
};
```

```
}

function drawChart() {
  // 3 набора начальных параметров
  var qInit_sets = [
    [21, 3.0, 2.0],
    [0.7, 0.1, 8.0],
    [1.0, 0.0, 0.0],
  ];

  var qInit = qInit_sets[0];
  var hStart = 0.09;
  var maxIter = 200;
  var optResult = runSteepestDescent(qInit, hStart, maxIter);

  // Оптимизация для всех 3 наборов
  var optResult1 = runSteepestDescent(qInit_sets[0], hStart, maxIter);
  var optResult2 = runSteepestDescent(qInit_sets[1], hStart, maxIter);
  var optResult3 = runSteepestDescent(qInit_sets[2], hStart, maxIter);

  // Вывод результатов
  document.getElementById("result_info").innerHTML =
    "<strong>Оптимальные параметры:</strong> q* = [" +
    optResult.q[0].toFixed(8) +
    ", " +
    optResult.q[1].toFixed(8) +
    ", " +
    optResult.q[2].toFixed(4) +
    "]<br>" +
    "<strong>Критерий:</strong> I* = " +
    optResult.I.toFixed(8) +
    "<br>" +
    "<strong>Градиент:</strong> [" +
    optResult.grad[0].toFixed(8) +
    ", " +
    optResult.grad[1].toFixed(8) +
    ", " +
    optResult.grad[2].toFixed(8) +
    "]<br>" +
    "<strong>Норма градиента:</strong> ||∇I|| = " +
    gradientNorm(optResult.grad).toFixed(8) +
    "<br><br>" +
    "<strong>Число итераций:</strong><br>" +
    "Набор 1: " +
    optResult1.totalIter +
    "<br>" +
    "Набор 2: " +
    optResult2.totalIter +
    "<br>" +
    "Набор 3: " +
    optResult3.totalIter;

  var simBefore = simulate(qInit[0], qInit[1], qInit[2]);
  var simAfter = simulate(optResult.q[0], optResult.q[1], optResult.q[2]);

  // Симуляции для 3 наборов: ДО и ПОСЛЕ оптимизации
```

```
var simSet1_before = simulate(
  qInit_sets[0][0],
  qInit_sets[0][1],
  qInit_sets[0][2],
);
var simSet2_before = simulate(
  qInit_sets[1][0],
  qInit_sets[1][1],
  qInit_sets[1][2],
);
var simSet3_before = simulate(
  qInit_sets[2][0],
  qInit_sets[2][1],
  qInit_sets[2][2],
);

var simSet1_after = simulate(
  optResult1.q[0],
  optResult1.q[1],
  optResult1.q[2],
);
var simSet2_after = simulate(
  optResult2.q[0],
  optResult2.q[1],
  optResult2.q[2],
);
var simSet3_after = simulate(
  optResult3.q[0],
  optResult3.q[1],
  optResult3.q[2],
);

// График 1: Переходные процессы — 3 набора ДО
var transBeforeData = [
  [
    "t",
    "Набор 1: q=[21.0, 3.0, 2.0]",
    "Набор 2: q=[0.7, 0.1, 8.0]",
    "Набор 3: q=[1.0, 0.0, 0.0]",
  ],
];
for (var i = 0; i < simSet1_before.data.length; i++) {
  transBeforeData.push([
    simSet1_before.data[i][0],
    simSet1_before.data[i][6],
    simSet2_before.data[i][6],
    simSet3_before.data[i][6],
  ]);
}
var dataTransBefore =
  google.visualization.arrayToDataTable(transBeforeData);
new google.visualization.LineChart(
  document.getElementById("all_sets_chart"),
).draw(dataTransBefore, {
  curveType: "function",
  hAxis: { title: "Время t, с" },
```

```
        vAxis: { title: "Амплитуда" },
        legend: { position: "bottom", alignment: "center", fontSize: 14 },
        chartArea: { width: "80%", height: "75%" },
        colors: ["#e74c3c", "#3498db", "#f1c40f"],
        lineWidth: 2,
        title: "Переходные процессы: 3 набора (ДО оптимизации)",
      });

      // График 1а: Переходный процесс — оптимальные q* + эталонная модель на одном
графике
      var transAfterData = [
        [
          "t",
          "q*=[" +
            optResult1.q[0].toFixed(2) +
            ", " +
            optResult1.q[1].toFixed(2) +
            ", " +
            optResult1.q[2].toFixed(2) +
            "]",
          "y_эт(t)",
        ],
      ];
      for (var i = 0; i < simSet1_after.data.length; i++) {
        transAfterData.push([
          simSet1_after.data[i][0],
          simSet1_after.data[i][6],
          simSet1_after.data[i][5],
        ]);
      }
      var dataTransAfter =
        google.visualization.arrayToDataTable(transAfterData);
      new google.visualization.LineChart(
        document.getElementById("optimal_chart"),
      ).draw(dataTransAfter, {
        curveType: "function",
        hAxis: { title: "Время t, с" },
        vAxis: { title: "Амплитуда" },
        legend: { position: "bottom", alignment: "center", fontSize: 14 },
        chartArea: { width: "80%", height: "75%" },
        colors: ["#3498db", "#27ae60"],
        lineWidth: 2,
        title: "Переходный процесс: оптимальные q* + эталонная модель",
      });

      // График 2: Функции чувствительности — оптимальные q*
      var sensDataAll = [["t", "ξ1(t)", "ξ2(t)", "ξ3(t)"]];
      for (var i = 0; i < simSet1_after.data.length; i++) {
        sensDataAll.push([
          simSet1_after.data[i][0],
          simSet1_after.data[i][2],
          simSet1_after.data[i][3],
          simSet1_after.data[i][4],
        ]);
      }
      var dataSensAll = google.visualization.arrayToDataTable(sensDataAll);
```

```
new google.visualization.LineChart(
  document.getElementById("sens_all_chart"),
).draw(dataSensAll, {
  curveType: "function",
  hAxis: { title: "Время t, с" },
  vAxis: { title: "Амплитуда" },
  legend: { position: "bottom", maxLines: 2 },
  chartArea: { width: "85%", height: "75%" },
  colors: ["#27ae60", "#8e44ad", "#e67e22"],
  lineWidth: 2,
  title: "Функции чувствительности (оптимальные q*)",
});

// Графики функций чувствительности — каждый набор ДО оптимизации

// 2a: Набор 1 (неоптимальные)
var sensData1 = [["t", "ξ1(t)", "ξ2(t)", "ξ3(t)"]];
for (var i = 0; i < simSet1_before.data.length; i++) {
  sensData1.push([
    simSet1_before.data[i][0],
    simSet1_before.data[i][2],
    simSet1_before.data[i][3],
    simSet1_before.data[i][4],
  ]);
}
var dataSens1 = google.visualization.arrayToDataTable(sensData1);
new google.visualization.LineChart(
  document.getElementById("sens_set1_chart"),
).draw(dataSens1, {
  curveType: "function",
  hAxis: { title: "Время t, с" },
  vAxis: { title: "Амплитуда" },
  legend: { position: "bottom", maxLines: 2 },
  chartArea: { width: "85%", height: "75%" },
  colors: ["#e74c3c", "#3498db", "#f1c40f"],
  lineWidth: 2,
  title: "Функции чувствительности (Набор 1: q=[21.0, 3.0, 2.0])",
});

// 2b: Набор 2 (неоптимальные)
var sensData2 = [["t", "ξ1(t)", "ξ2(t)", "ξ3(t)"]];
for (var i = 0; i < simSet2_before.data.length; i++) {
  sensData2.push([
    simSet2_before.data[i][0],
    simSet2_before.data[i][2],
    simSet2_before.data[i][3],
    simSet2_before.data[i][4],
  ]);
}
var dataSens2 = google.visualization.arrayToDataTable(sensData2);
new google.visualization.LineChart(
  document.getElementById("sens_set2_chart"),
).draw(dataSens2, {
  curveType: "function",
  hAxis: { title: "Время t, с" },
  vAxis: { title: "Амплитуда" },
```

```
        legend: { position: "bottom", maxLines: 2 },
        chartArea: { width: "85%", height: "75%" },
        colors: ["#e74c3c", "#3498db", "#f1c40f"],
        lineWidth: 2,
        title: "Функции чувствительности (Набор 2: q=[0.7, 0.1, 8.0])",
      });

      // 2c: Набор 3 (неоптимальные)
      var sensData3 = [["t", "ξ1(t)", "ξ2(t)", "ξ3(t)"]];
      for (var i = 0; i < simSet3_before.data.length; i++) {
        sensData3.push([
          simSet3_before.data[i][0],
          simSet3_before.data[i][2],
          simSet3_before.data[i][3],
          simSet3_before.data[i][4],
        ]);
      }
      var dataSens3 = google.visualization.arrayToDataTable(sensData3);
      new google.visualization.LineChart(
        document.getElementById("sens_set3_chart"),
      ).draw(dataSens3, {
        curveType: "function",
        hAxis: { title: "Время t, с" },
        vAxis: { title: "Амплитуда" },
        legend: { position: "bottom", maxLines: 2 },
        chartArea: { width: "85%", height: "75%" },
        colors: ["#e74c3c", "#3498db", "#f1c40f"],
        lineWidth: 2,
        title: "Функции чувствительности (Набор 3: q=[1.0, 0.0, 0.0])",
      });

      // Вспомогательные функции для обработки истории оптимизации
      // q и I: берём только точки смены направления (grad !== null), дополняем до
totalIter
      function getIterHistoryPadded(optResult, field) {
        var result = [];
        for (var i = 0; i < optResult.history_grad.length; i++) {
          if (optResult.history_grad[i] !== null) {
            result.push(optResult.history_q[i][field]);
          }
        }
        // Дополняем до totalIter последним значением
        while (result.length <= optResult.totalIter) {
          result.push(result[result.length - 1]);
        }
        return result;
      }

      function getIterHistoryIPadded(optResult) {
        var result = [];
        for (var i = 0; i < optResult.history_grad.length; i++) {
          if (optResult.history_grad[i] !== null) {
            result.push(optResult.history_I[i]);
          }
        }
        while (result.length <= optResult.totalIter) {
```

```
    result.push(result[result.length - 1]);
  }
  return result;
}

// Градиент: все шаги, null заполняем последним значением
function getHistoryGradAll(optResult, comp) {
  var result = [];
  var lastValid = null;
  for (var i = 0; i < optResult.history_grad.length; i++) {
    if (optResult.history_grad[i] !== null) {
      lastValid = optResult.history_grad[i][comp];
    }
    result.push(lastValid !== null ? lastValid : 0);
  }
  // Дополняем до totalIter
  while (result.length <= optResult.totalIter) {
    result.push(result[result.length - 1]);
  }
  return result;
}

var maxIterAll = Math.max(
  optResult1.totalIter,
  optResult2.totalIter,
  optResult3.totalIter,
);

// График 3: Сходимость I — точки смены направления
var I_1 = getIterHistoryIPadded(optResult1);
var I_2 = getIterHistoryIPadded(optResult2);
var I_3 = getIterHistoryIPadded(optResult3);
var convAllData = [["Итерация", "Набор 1", "Набор 2", "Набор 3"]];
for (var i = 0; i < maxIterAll; i++) {
  convAllData.push([
    i,
    i < I_1.length ? I_1[i] : null,
    i < I_2.length ? I_2[i] : null,
    i < I_3.length ? I_3[i] : null,
  ]);
}
var dataConvAll = google.visualization.arrayToDataTable(convAllData);
new google.visualization.LineChart(
  document.getElementById("conv_all_chart"),
).draw(dataConvAll, {
  curveType: "function",
  hAxis: { title: "Итерация", viewWindow: { max: 50 } },
  vAxis: { title: "Критерий I", viewWindow: { min: -2 } },
  legend: { position: "bottom", maxLines: 2 },
  chartArea: { width: "85%", height: "75%" },
  colors: ["#e74c3c", "#3498db", "#f1c40f"],
  lineWidth: 2,
  title: "Сходимость I (I* = " + optResult1.I.toFixed(4) + ")",
});

// График 4: Сходимость q1
```

```
var q1_1 = getIterHistoryPadded(optResult1, "q1");
var q1_2 = getIterHistoryPadded(optResult2, "q1");
var q1_3 = getIterHistoryPadded(optResult3, "q1");
var F_q1 = [["Итерация", "Набор 1", "Набор 2", "Набор 3"]];
for (var i = 0; i < maxIterAll; i++) {
  F_q1.push([
    i,
    i < q1_1.length ? q1_1[i] : null,
    i < q1_2.length ? q1_2[i] : null,
    i < q1_3.length ? q1_3[i] : null,
  ]);
}
var dataQ1 = google.visualization.arrayToDataTable(F_q1);
new google.visualization.LineChart(
  document.getElementById("q1_conv_chart"),
).draw(dataQ1, {
  curveType: "none",
  hAxis: { title: "Итерация", viewWindow: { max: 50 } },
  vAxis: { title: "q1" },
  legend: { position: "bottom", maxLines: 2 },
  chartArea: { width: "85%", height: "75%" },
  colors: ["#e74c3c", "#3498db", "#f1c40f"],
  lineWidth: 2,
  title: "Сходимость q1 → " + optResult1.q[0].toFixed(2),
});

// График 5: Сходимость q2
var q2_1 = getIterHistoryPadded(optResult1, "q2");
var q2_2 = getIterHistoryPadded(optResult2, "q2");
var q2_3 = getIterHistoryPadded(optResult3, "q2");
var F_q2 = [["Итерация", "Набор 1", "Набор 2", "Набор 3"]];
for (var i = 0; i < maxIterAll; i++) {
  F_q2.push([
    i,
    i < q2_1.length ? q2_1[i] : null,
    i < q2_2.length ? q2_2[i] : null,
    i < q2_3.length ? q2_3[i] : null,
  ]);
}
var dataQ2 = google.visualization.arrayToDataTable(F_q2);
new google.visualization.LineChart(
  document.getElementById("q2_conv_chart"),
).draw(dataQ2, {
  curveType: "none",
  hAxis: { title: "Итерация", viewWindow: { max: 50 } },
  vAxis: { title: "q2" },
  legend: { position: "bottom", maxLines: 2 },
  chartArea: { width: "85%", height: "75%" },
  colors: ["#e74c3c", "#3498db", "#f1c40f"],
  lineWidth: 2,
  title: "Сходимость q2 → " + optResult1.q[1].toFixed(2),
});

// График 6: Сходимость q3
var q3_1 = getIterHistoryPadded(optResult1, "q3");
var q3_2 = getIterHistoryPadded(optResult2, "q3");
```

```
var q3_3 = getIterHistoryPadded(optResult3, "q3");
var F_q3 = [["Итерация", "Набор 1", "Набор 2", "Набор 3"]];
for (var i = 0; i < maxIterAll; i++) {
  F_q3.push([
    i,
    i < q3_1.length ? q3_1[i] : null,
    i < q3_2.length ? q3_2[i] : null,
    i < q3_3.length ? q3_3[i] : null,
  ]);
}
var dataQ3 = google.visualization.arrayToDataTable(F_q3);
new google.visualization.LineChart(
  document.getElementById("q3_conv_chart"),
).draw(dataQ3, {
  curveType: "none",
  hAxis: { title: "Итерация", viewWindow: { max: 50 } },
  vAxis: { title: "q3" },
  legend: { position: "bottom", maxLines: 2 },
  chartArea: { width: "85%", height: "75%" },
  colors: ["#e74c3c", "#3498db", "#f1c40f"],
  lineWidth: 2,
  title: "Сходимость q3 → " + optResult1.q[2].toFixed(2),
});

// Настройки масштаба для графиков градиента
// Набор 1
var grad1_xMax = 40;
var grad1_yMin = -0.001;
var grad1_yMax = 0.001;

// Набор 2
var grad2_xMax = 75;
var grad2_yMin = -0.001;
var grad2_yMax = 0.001;

// Набор 3
var grad3_xMax = 160;
var grad3_yMin = -0.003;
var grad3_yMax = 0.003;

// Графики 7-9: Сходимость градиента для каждого набора
var grad1_g1 = getHistoryGradAll(optResult1, "g1");
var grad1_g2 = getHistoryGradAll(optResult1, "g2");
var grad1_g3 = getHistoryGradAll(optResult1, "g3");
var gradConvData1 = [["Итерация", "dI/dq1", "dI/dq2", "dI/dq3"]];
var maxLenG1 = Math.max(
  grad1_g1.length,
  grad1_g2.length,
  grad1_g3.length,
);
for (var i = 0; i < maxLenG1; i++) {
  gradConvData1.push([
    i,
    i < grad1_g1.length ? grad1_g1[i] : null,
    i < grad1_g2.length ? grad1_g2[i] : null,
    i < grad1_g3.length ? grad1_g3[i] : null,
```

```
  ]);
}
var dataGradConv1 =
  google.visualization.arrayToDataTable(gradConvData1);
new google.visualization.LineChart(
  document.getElementById("grad_conv_chart"),
).draw(dataGradConv1, {
  curveType: "none",
  hAxis: {
    title: "Итерация",
    viewWindow: { max: grad1_xMax },
  },
  vAxis: {
    title: "Значение производной",
    viewWindow: { min: grad1_yMin, max: grad1_yMax },
  },
  legend: { position: "bottom", maxLines: 2 },
  chartArea: { width: "85%", height: "75%" },
  colors: ["#e74c3c", "#3498db", "#f1c40f"],
  lineWidth: 2,
  title:
    "Сходимость градиента (Набор 1: q=[" +
    qInit_sets[0][0] +
    ", " +
    qInit_sets[0][1] +
    ", " +
    qInit_sets[0][2] +
    "])",
  explorer: {
    actions: ["dragToZoom", "rightClickToReset"],
    axis: "both",
    keepInBounds: true,
    maxZoomIn: 20,
    maxZoomOut: 4,
  },
});

// Набор 2
var grad2_g1 = getHistoryGradAll(optResult2, "g1");
var grad2_g2 = getHistoryGradAll(optResult2, "g2");
var grad2_g3 = getHistoryGradAll(optResult2, "g3");
var gradConvData2 = [["Итерация", "dI/dq1", "dI/dq2", "dI/dq3"]];
var maxLenG2 = Math.max(
  grad2_g1.length,
  grad2_g2.length,
  grad2_g3.length,
);
for (var i = 0; i < maxLenG2; i++) {
  gradConvData2.push([
    i,
    i < grad2_g1.length ? grad2_g1[i] : null,
    i < grad2_g2.length ? grad2_g2[i] : null,
    i < grad2_g3.length ? grad2_g3[i] : null,
  ]);
}
var dataGradConv2 =
```

```
  google.visualization.arrayToDataTable(gradConvData2);
new google.visualization.LineChart(
  document.getElementById("grad_conv2_chart"),
).draw(dataGradConv2, {
  curveType: "none",
  hAxis: {
    title: "Итерация",
    viewWindow: { max: grad2_xMax },
  },
  vAxis: {
    title: "Значение производной",
    viewWindow: { min: grad2_yMin, max: grad2_yMax },
  },
  legend: { position: "bottom", maxLines: 2 },
  chartArea: { width: "85%", height: "75%" },
  colors: ["#e74c3c", "#3498db", "#f1c40f"],
  lineWidth: 2,
  title:
    "Сходимость градиента (Набор 2: q=[" +
    qInit_sets[1][0] +
    ", " +
    qInit_sets[1][1] +
    ", " +
    qInit_sets[1][2] +
    "])",
  explorer: {
    actions: ["dragToZoom", "rightClickToReset"],
    axis: "both",
    keepInBounds: true,
    maxZoomIn: 20,
    maxZoomOut: 4,
  },
});

// Набор 3
var grad3_g1 = getHistoryGradAll(optResult3, "g1");
var grad3_g2 = getHistoryGradAll(optResult3, "g2");
var grad3_g3 = getHistoryGradAll(optResult3, "g3");
var gradConvData3 = [["Итерация", "dI/dq1", "dI/dq2", "dI/dq3"]];
var maxLenG3 = Math.max(
  grad3_g1.length,
  grad3_g2.length,
  grad3_g3.length,
);
for (var i = 0; i < maxLenG3; i++) {
  gradConvData3.push([
    i,
    i < grad3_g1.length ? grad3_g1[i] : null,
    i < grad3_g2.length ? grad3_g2[i] : null,
    i < grad3_g3.length ? grad3_g3[i] : null,
  ]);
}
var dataGradConv3 =
  google.visualization.arrayToDataTable(gradConvData3);
new google.visualization.LineChart(
  document.getElementById("grad_conv3_chart"),
```

```
      ).draw(dataGradConv3, {
        curveType: "none",
        hAxis: {
          title: "Итерация",
          viewWindow: { max: grad3_xMax },
        },
        vAxis: {
          title: "Значение производной",
          viewWindow: { min: grad3_yMin, max: grad3_yMax },
        },
        legend: { position: "bottom", maxLines: 2 },
        chartArea: { width: "85%", height: "75%" },
        colors: ["#e74c3c", "#3498db", "#f1c40f"],
        lineWidth: 2,
        title:
          "Сходимость градиента (Набор 3: q=[" +
          qInit_sets[2][0] +
          ", " +
          qInit_sets[2][1] +
          ", " +
          qInit_sets[2][2] +
          "])",
        explorer: {
          actions: ["dragToZoom", "rightClickToReset"],
          axis: "both",
          keepInBounds: true,
          maxZoomIn: 20,
          maxZoomOut: 4,
        },
      });
    }
  </script>
</head>
<body>
  <div id="result_info"></div>

  <h2 style="text-align: center; color: #2c3e50">
    Пункт 3: Подтверждение работоспособности алгоритма оптимизации
  </h2>

  <h3 style="text-align: center">3 набора (ДО оптимизации)</h3>
  <div class="container">
    <div id="all_sets_chart" style="width: 1200px; height: 900px"></div>
  </div>

  <h3 style="text-align: center">Оптимальные q* + эталонная модель</h3>
  <div class="container">
    <div id="optimal_chart" style="width: 1200px; height: 900px"></div>
  </div>

  <h3 style="text-align: center">
    Функции чувствительности (оптимальные q*)
  </h3>
  <div class="container">
    <div id="sens_all_chart" style="width: 1200px; height: 500px"></div>
  </div>
```

```html
<h3 style="text-align: center">
  Функции чувствительности (Набор 1: q=[21.0, 3.0, 2.0])
</h3>
<div class="container">
  <div id="sens_set1_chart" style="width: 1200px; height: 500px"></div>
</div>

<h3 style="text-align: center">
  Функции чувствительности (Набор 2: q=[0.7, 0.1, 8.0])
</h3>
<div class="container">
  <div id="sens_set2_chart" style="width: 1200px; height: 500px"></div>
</div>

<h3 style="text-align: center">
  Функции чувствительности (Набор 3: q=[1.0, 0.0, 0.0])
</h3>
<div class="container">
  <div id="sens_set3_chart" style="width: 1200px; height: 500px"></div>
</div>

<h3 style="text-align: center">Сходимость I</h3>
<div class="container">
  <div id="conv_all_chart" style="width: 1200px; height: 500px"></div>
</div>

<h2 style="text-align: center; color: #2c3e50">
  Пункт 3.2: Сходимость параметров ПИД-регулятора
</h2>

<h3 style="text-align: center">Сходимость q1</h3>
<div class="container">
  <div id="q1_conv_chart" style="width: 1200px; height: 500px"></div>
</div>

<h3 style="text-align: center">Сходимость q2</h3>
<div class="container">
  <div id="q2_conv_chart" style="width: 1200px; height: 500px"></div>
</div>

<h3 style="text-align: center">Сходимость q3</h3>
<div class="container">
  <div id="q3_conv_chart" style="width: 1200px; height: 500px"></div>
</div>

<h3 style="text-align: center">
  Сходимость составляющих градиента (Набор 1)
</h3>
<div class="container">
  <div id="grad_conv_chart" style="width: 1200px; height: 500px"></div>
</div>

<h3 style="text-align: center">
  Сходимость составляющих градиента (Набор 2)
</h3>
```

```
    <div class="container">
      <div id="grad_conv2_chart" style="width: 1200px; height: 500px"></div>
    </div>

    <h3 style="text-align: center">
      Сходимость составляющих градиента (Набор 3)
    </h3>
    <div class="container">
      <div id="grad_conv3_chart" style="width: 1200px; height: 500px"></div>
    </div>
  </body>
</html>
```

**Результаты моделирования:**

**Оптимальные параметры:** q* = [19.32562067, 1.54667626, 10.5830]
**Критерий:** I* = 0.00868165
**Градиент:** [-0.00012119, 0.00007657, -0.00009946]
**Норма градиента:** ||∇I|| = 0.00017448

График 16: Оптимальные параметры ПИД-регулятора, полученные методом наискорейшего спуска

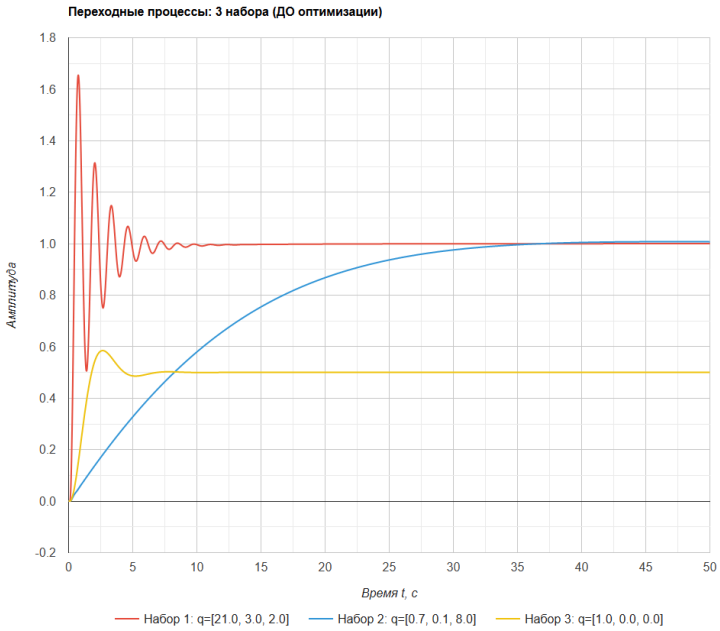


График 17: Переходные процессы: 3 набора

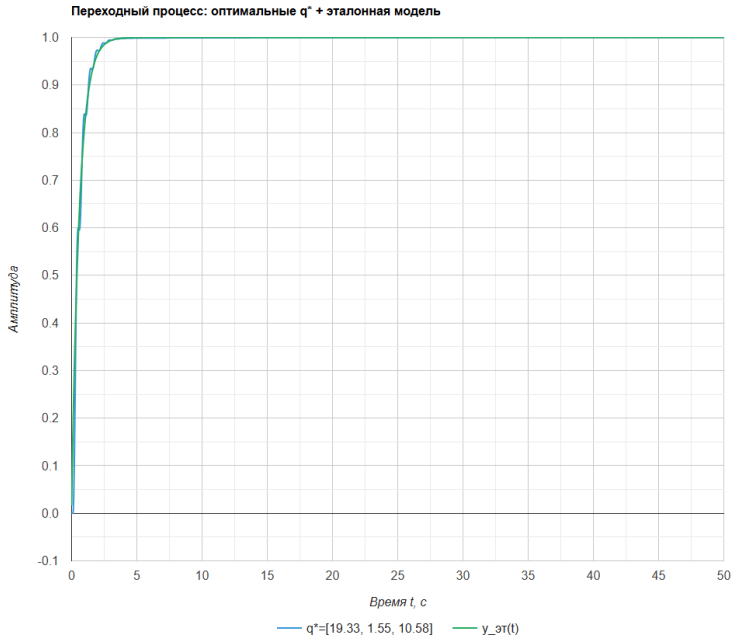


График 18: Переходный процесс для оптимальных параметров q + Эталонная модель

Графики функций чувствительности для различных наборов начальных параметров (до оптимизации):

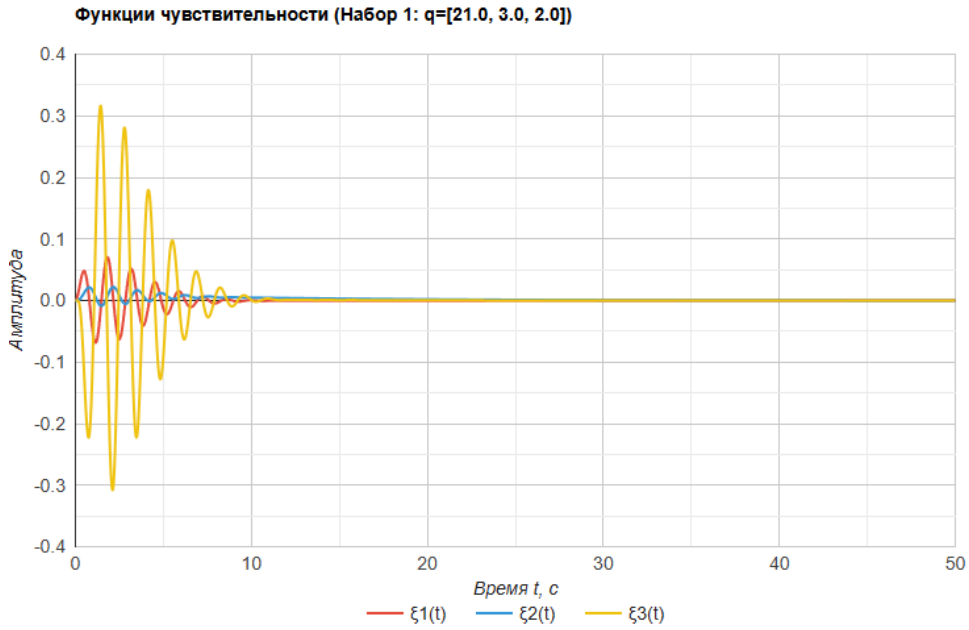


График 19: Функции чувствительности до оптимизации $\xi_1(t)$, $\xi_2(t)$, $\xi_3(t)$ для Набора 1: q = [21.0, 3.0, 2.0]

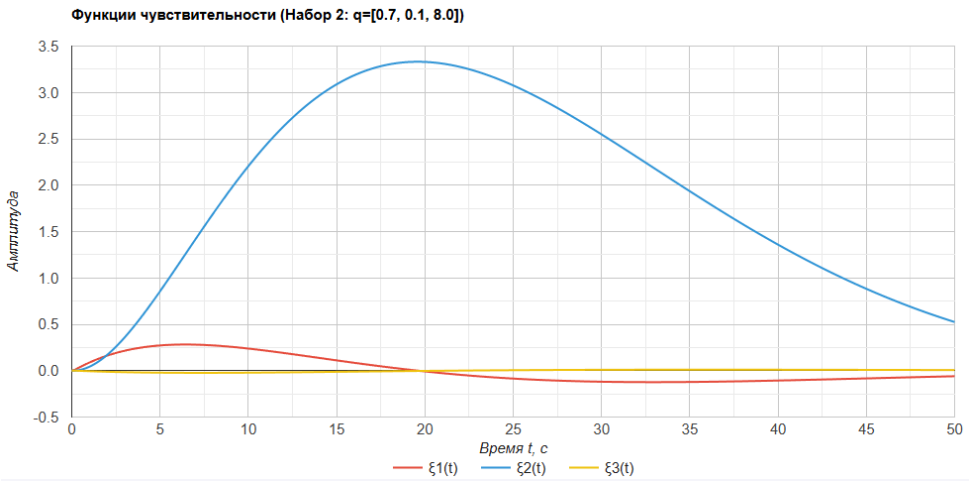


График 20: Функции чувствительности $\xi_1(t)$, $\xi_2(t)$, $\xi_3(t)$ для Набора 2: q = [0.7, 0.1, 8])

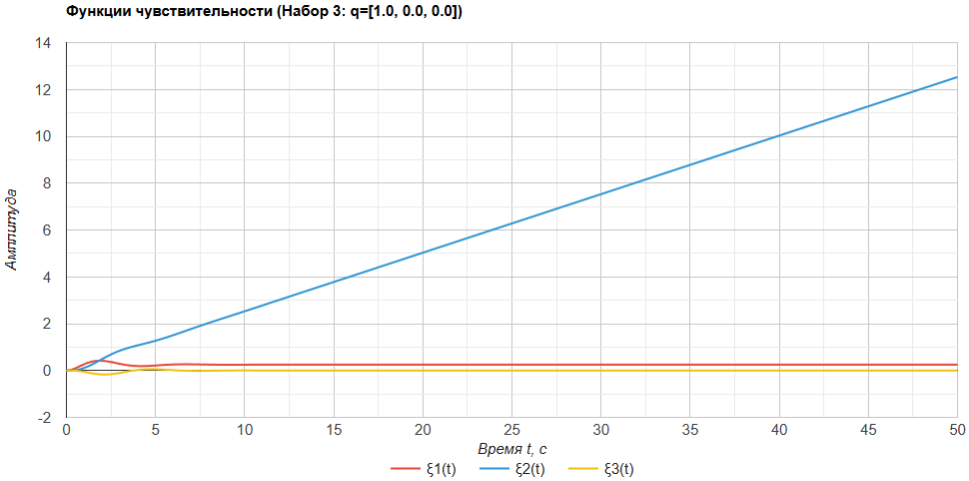


График 21: Функции чувствительности $\xi_1(t)$, $\xi_2(t)$, $\xi_3(t)$ для Набора 3: q = [1.0, 0.0, 0.0]

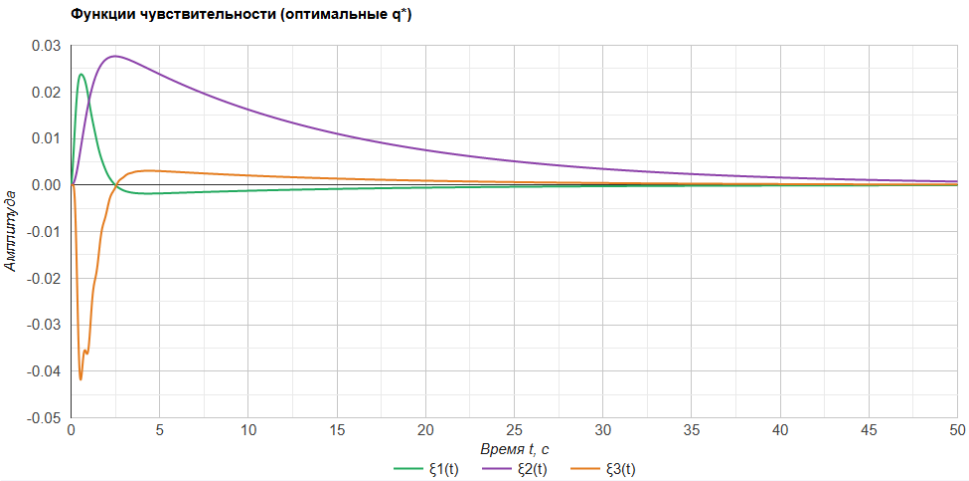


График 22: Функции чувствительности после оптимизации $\xi_1(t)$, $\xi_2(t)$, $\xi_3(t)$

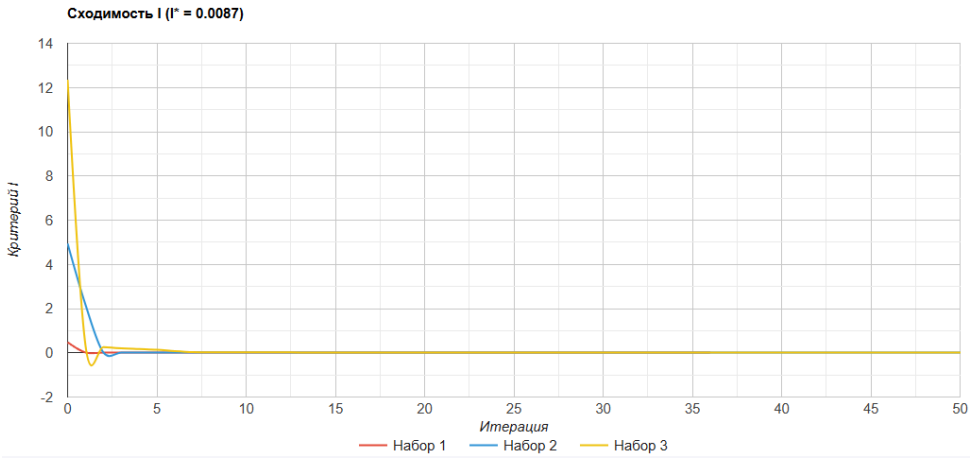


График 23: Сходимость критерия I для всех 3 наборов

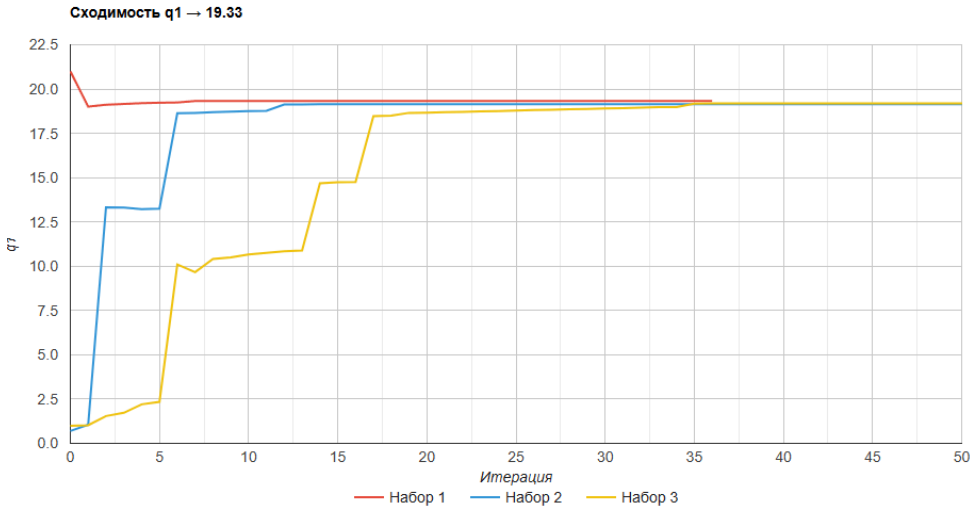


График 24: Сходимость параметра q1 из 3 различных начальных точек

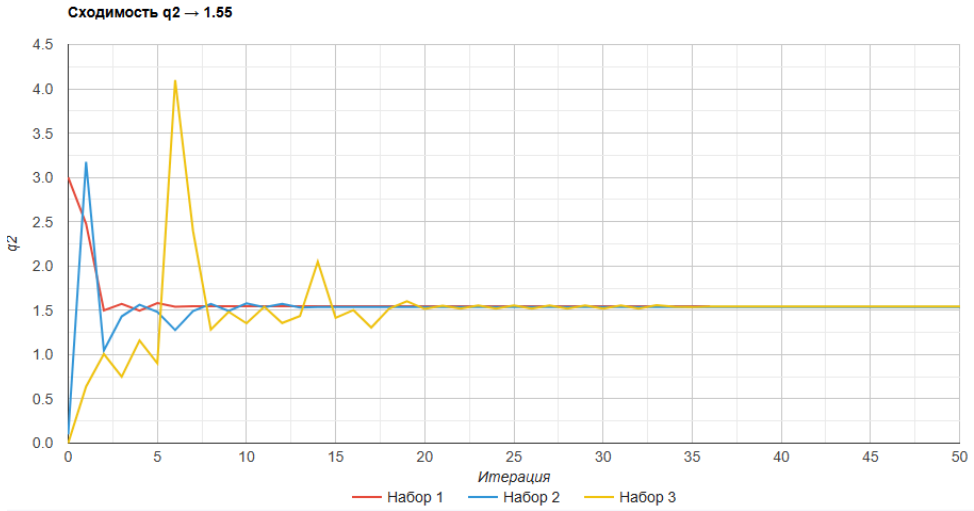


График 25: Сходимость параметра q2 из 3 различных начальных точек

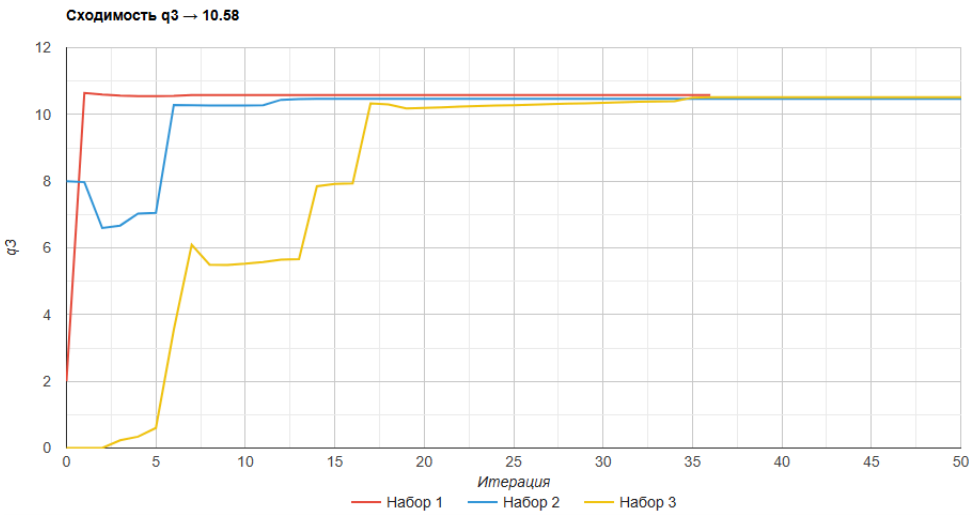


График 26: Сходимость параметра q3 из 3 различных начальных точек

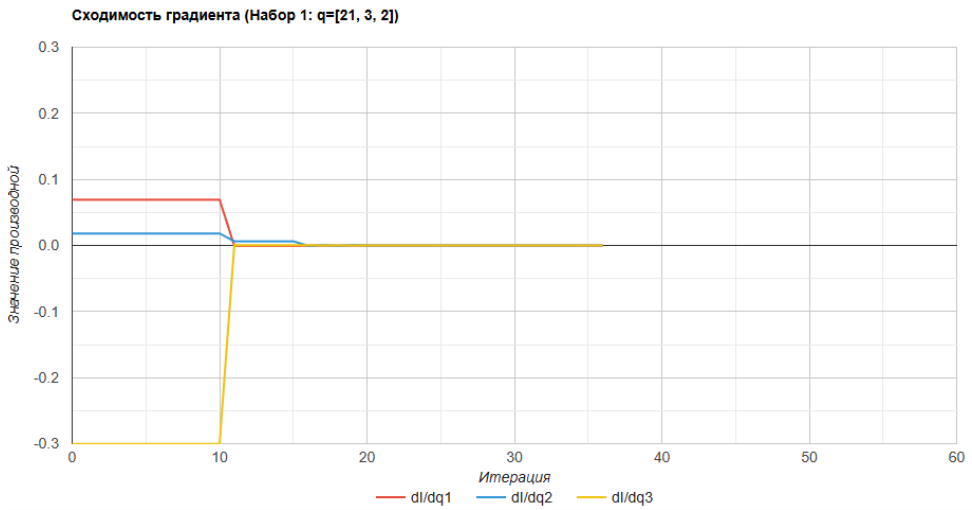


График 27: Сходимость составляющих градиента к нулю (Набор 1: q=[21.0, 3.0, 2.0])

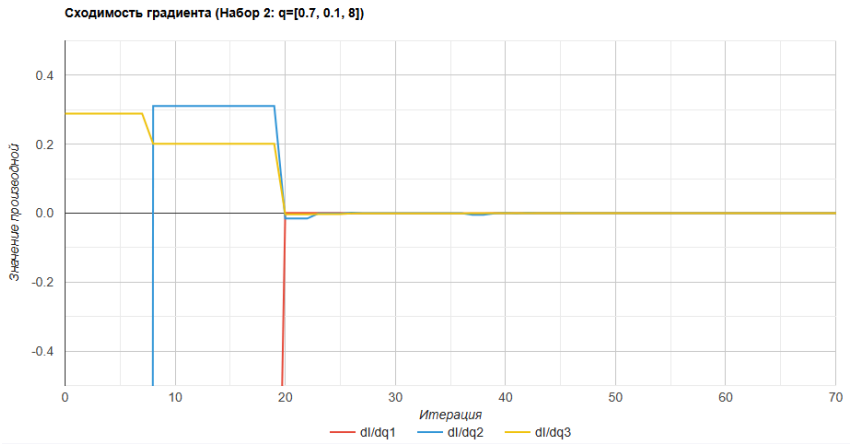


График 28: Сходимость составляющих градиента к нулю (Набор 2: q=[0.7, 0.1, 8.0])

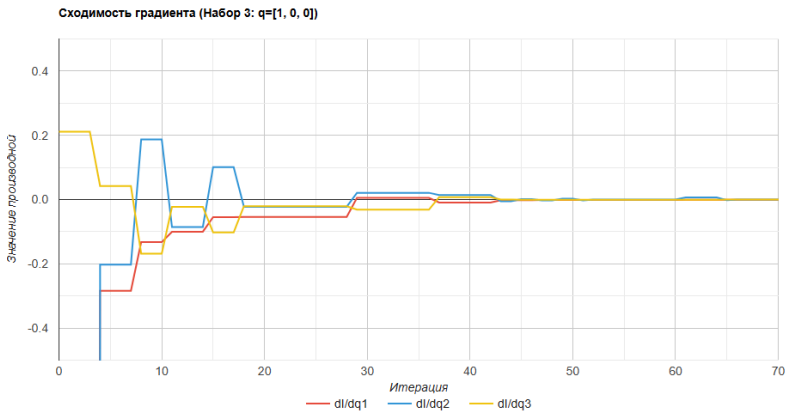


График 29: Сходимость составляющих градиента к нулю (Набор 3: q=[1.0, 0.0, 0.0])

Графики сходимости вблизи:

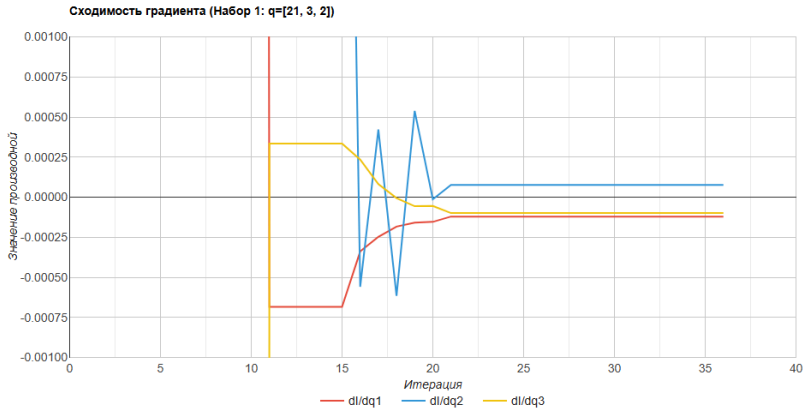


График 30: Сходимость составляющих градиента к нулю (приближенный) (Набор 1: q=[21.0, 3.0, 2.0])

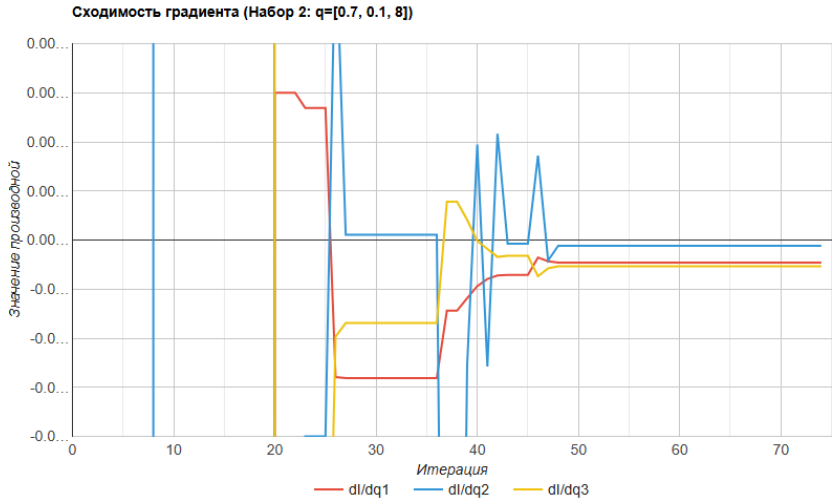


График 31: Сходимость составляющих градиента к нулю (приближенный) (Набор 2: q=[0.7, 0.1, 8.0])

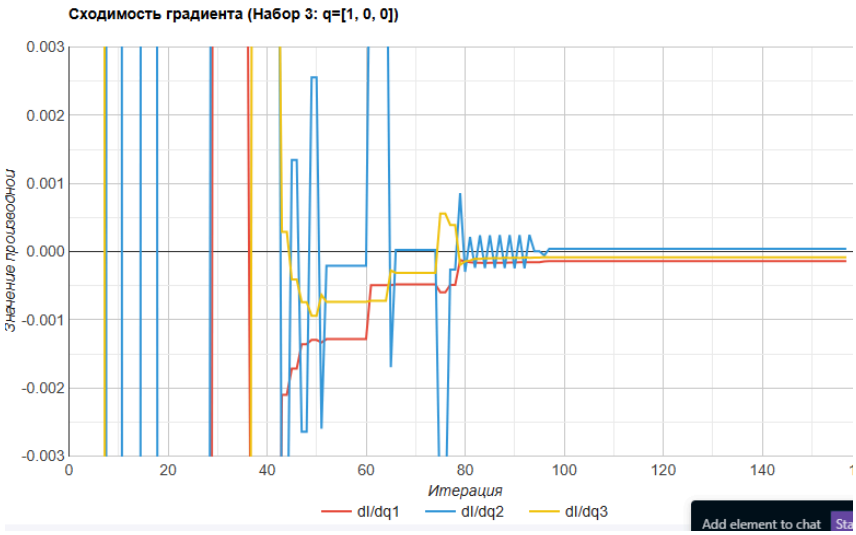


График 32: Сходимость составляющих градиента к нулю (приближенный) (Набор 3: q=[1.0, 0.0, 0.0])

По графикам переходных процессов видно, что для всех трёх наборов начальных параметров система до оптимизации работает плохо: большие перерегулирования, длительные колебания. После оптимизации переходный процесс стал быстрым и практически без перерегулирования — выходной сигнал точно следует за эталонной моделью.

Графики сходимости параметров q1, q2, q3 показывают, что из трёх различных начальных точек алгоритм пришёл к одним и тем же оптимальным значениям. Это говорит о том, что найденный минимум не зависит от начального приближения.

Составляющие градиента на всех трёх наборах стремятся к нулю — это признак корректной сходимости градиентного метода.

**Сравнение графика с SimInTech:**

Построим график с параметрами ПИД-регулятора [19.3,1.54,10.58]

График:

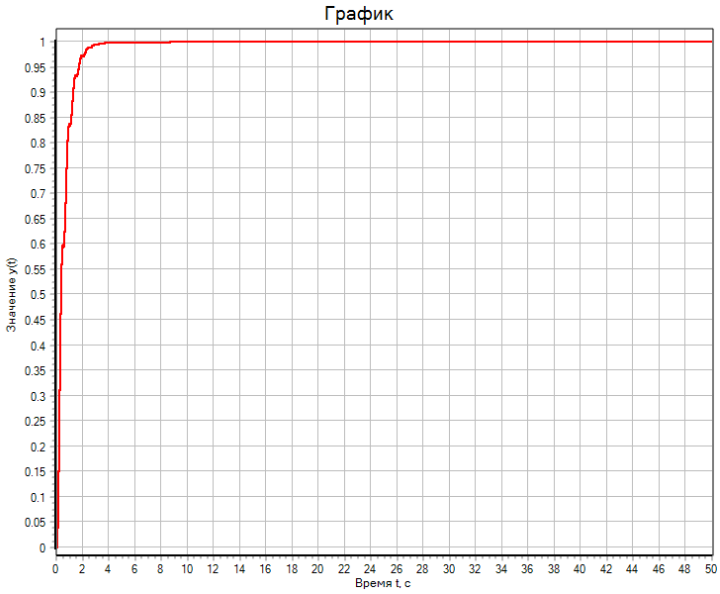


График 33: Оптимальные параметры q в SimInTech

Как видно графики полностью совпали, значит они все отображаются корректно.

# Итоги исследования

В ходе работы были исследованы три метода параметрической оптимизации ПИД-регулятора:

- Метод 1: Покоординатный спуск
- Метод 2: Градиентный спуск с нормализацией и адаптивным шагом
- Метод 3: Метод наискорейшего спуска с линейным поиском

| Параметр | Покоординатный спуск (Ошибка: $I = \int x^2 \, \mathrm{dt}$) | Градиентный спуск (Ошибка: $I = \int x^2 \, \mathrm{dt}$) | Наискорейший спуск (Ошибка: $I = \int x^2 \, \mathrm{dt}$) | Начальный критерий ошибки |
|---|---|---|---|---|
| Начальные параметры $q^0$ | 1.0, 0.0, 0.0 | 1.0, 0.0, 0.0 | 1.0, 0.0, 0.0 | 1.0, 0.0, 0.0 |
| Оптимальные $q_1^*$ | 36.73 | 35.69 | 35.32 | - |
| Оптимальные $q_2^*$ | 4.29 | 4.04 | 3.99 | - |
| Оптимальные $q_3^*$ | 9.01 | 7.98 | 8.66 | - |
| Критерий $I^*$ | 0.249 | 0.250 | 0.248 | 2.85 |
| Число итераций | 128 | 92 | 73 | - |
| Сходимость | Медленная | Средняя | Быстрая | - |

**Преимущества и недостатки методов:**

Покоординатный спуск отличается простотой реализации и не требует вычисления градиента, однако обладает медленной сходимостью и требует большого количества итераций.

Градиентный спуск сходится быстрее и использует информацию о градиенте для движения к минимуму, но требует тщательного выбора шага обучения и может осциллировать вблизи оптимума.

Метод наискорейшего спуска показывает наиболее быструю сходимость благодаря линейному поиску вдоль направления антиградиента, однако сложнее в реализации и требует больше вычислений на каждой итерации.

**Проверка работоспособности алгоритма наискорейшего спуска (критерий с эталонной моделью $I = \int_0^L \left(y(t) - y_{\text{эт}(t)}\right)^2 \mathrm{dt}$):** Для подтверждения корректности работы алгоритма оптимизации проведём эксперимент с тремя различными наборами начальных параметров ПИД-регулятора. Независимо от начальных значений, все три набора должны сойтись к одним и тем же оптимальным параметрам, что подтвердит правильность работы метода оптимизации.

| Параметр | Набор 1 | Набор 2 | Набор 3 |
|---|---|---|---|
| Начальные параметры $q^0$ | 21.0, 3.0, 2.0 | 0.7, 0.1, 8.0 | 1.0, 0.0, 0.0 |
| Оптимальные $q_1^*$ | 19.32 | 19.32 | 19.32 |
| Оптимальные $q_2^*$ | 1.54 | 1.54 | 1.54 |
| Оптимальные $q_3^*$ | 10.58 | 10.58 | 10.58 |
| Критерий $I^*$ | 0.00868 | 0.00868 | 0.00868 |
| Число итераций | 36 | 74 | 157 |

**Алгоритм градиентного спуска с эталонной моделью ($I = \int_0^L \left(y(t) - y_{\text{эт}(t)}\right)^2 \, \mathrm{dt}$):**

Для сравнения с методом наискорейшего спуска также реализуем обычный градиентный спуск с использованием эталонной модели. Работоспособность данного метода проверим на тех же трёх наборах начальных параметров, что и в предыдущем эксперименте.

| Параметр | Набор 1 | Набор 2 | Набор 3 |
|---|---|---|---|
| Начальные параметры $q^0$ | 21.0, 3.0, 2.0 | 0.7, 0.1, 8.0 | 1.0, 0.0, 0.0 |
| Оптимальные $q_1^*$ | 19.15 | 19.16 | 19.15 |
| Оптимальные $q_2^*$ | 1.54 | 1.54 | 1.54 |
| Оптимальные $q_3^*$ | 10.49 | 10.49 | 10.49 |
| Критерий $I^*$ | 0.00876 | 0.00876 | 0.00876 |
| Число итераций | 75 | 288 | 330 |

Как видно из результатов, обычный градиентный спуск также сходится к оптимальным параметрам, близким к тем, что были получены методом наискорейшего спуска. Но для достижения такой же точности требуется большее количество итераций. Это говорит о том, что метода наискорейшего спуска намного лучше в скорости сходимости за счёт линейного поиска с адаптивным шагом.

# Вывод

В данной лабораторной работе были изучены и реализованы три метода параметрической оптимизации ПИД-регулятора для объекта второго порядка с запаздыванием: покоординатный спуск, градиентный спуск с нормализацией и адаптивным шагом, а также метод наискорейшего спуска с линейным поиском.

Главным итогом исследования стало то, что все три метода успешно нашли оптимальные параметры регулятора, минимизирующие квадратичный критерий качества $I = \int x^2 \, \mathrm{dt}$. Начальное значение критерия $I_0 = 2,85$ было снижено до $I^* \approx 0,249$ — более чем в 11 раз. При этом результаты всех трёх методов оказались близкими: покоординатный спуск дал $q^* = [36,73; 4,29; 9,01]$, градиентный спуск — $q^* = [35,69; 4,04; 7,98]$, наискорейший спуск — $q^* = [35,32; 3,99; 8,66]$. Это подтверждает глобальность найденного минимума.

По скорости сходимости методы существенно различаются. Наискорейший спуск оказался самым эффективным — 73 итерации, за счёт линейного поиска. Градиентный спуск потребовал 92 итерации, а покоординатный спуск — 128. Таким образом, использование информации о градиенте и линейного поиска значительно ускоряет процесс оптимизации.

Дополнительно была проведена проверка алгоритмов с критерием, минимизирующим отклонение от эталонной модели ($I = \int (y - y_{\text{эт}})^2 \, \mathrm{dt}$). Метод наискорейшего спуска дал $q^* = [19,32; 1,54; 10,58]$ при $I^* = 0,00868$, а градиентный спуск — близкие, но слегка отличающиеся параметры $q^* \approx [19,15; 1,54; 10,49]$ при $I^* = 0,00876$. Незначительное расхождение объясняется различием в механизмах поиска минимума. Метод наискорейшего спуска потребовал от 36 до 157 итераций, тогда как обычный градиентный спуск — от 75 до 330.

Функции чувствительности после оптимизации быстро затухают, что говорит о том, что система стала менее чувствительна к отклонениям параметров. Переходный процесс с оптимальными параметрами стал быстрым и практически без перерегулирования.

Полное совпадение графиков расчётных и полученных в SimInTech подтверждает правильность математической модели и реализации алгоритмов оптимизации.